ENGENHARIA DE SOFTWARE · UNIDADE 4

UML — Modelagem do Sistema

Cinco diagramas, cinco perguntas diferentes sobre o mesmo sistema.

Aulas 27 a 37 (8ª a 11ª semana) · CST em ADS · IFRO · 2026/2

ROTEIRO

O percurso da unidade

Aulas	Diagrama	Pergunta que responde
27	Panorama	Que diagramas existem, e quando cada um serve?
28	**Casos de uso**	Quem usa o sistema, e para quê?
29–31	**Classes**	Que conceitos existem no domínio, e como se ligam?
32	**Sequência**	Em que ordem os objetos conversam?
33	**Atividades**	Qual é o fluxo do processo?
34	**Estados**	Por quais situações um objeto passa?
35–37	Laboratório e revisão cruzada	Tudo isso é consistente entre si?

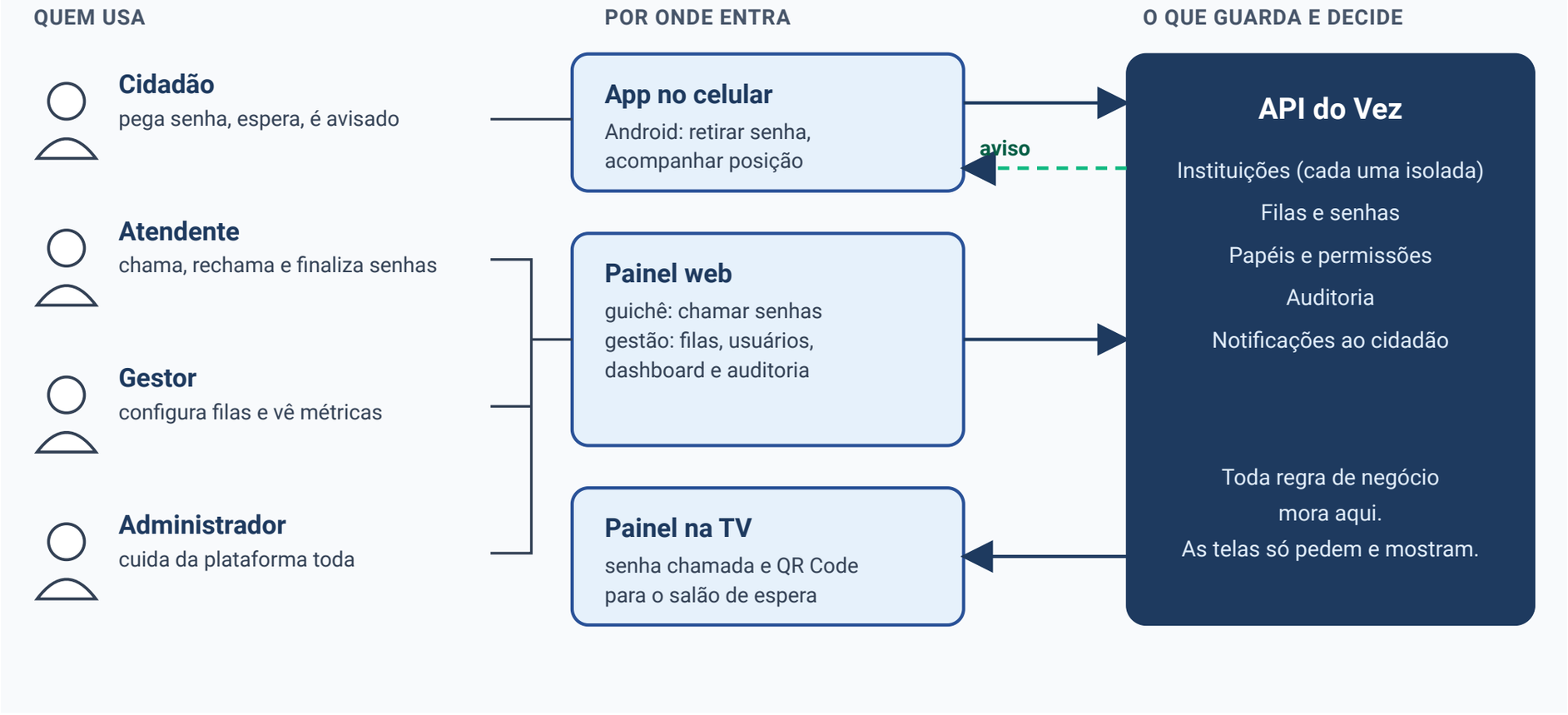
> No **Milestone 2**, os diagramas UML valem **20 dos 100 pontos** — e seguem valendo 20 no Milestone 3.

O CASO QUE ATRAVESSA A UNIDADE

Um sistema real: o Vez resolve a fila e a senha

O problema: filas físicas, senha de papel e ninguém sabe quando é a sua vez.

O Vez emite a senha (no app, por QR Code ou no guichê), organiza a fila da instituição e avisa quando chega a vez.



📖 Exemplo: projeto Vez — docs/01-visao-geral.md e docs/03-acesso-rbac.md (esboço livre, não é UML)

O CASO QUE ATRAVESSA A UNIDADE

O Vez dá requisitos reais aos cinco diagramas

Funcionais

- **V-RF-01** emitir senha
- **V-RF-02** chamar, rechamar e finalizar
- **V-RF-03** cancelar (só se aguardando)
- **V-RF-04** notificar o cidadão
- **V-RF-05** avaliar o atendimento

Não funcionais

- **V-RNF-01** cada instituição só vê os seus dados
- **V-RNF-02** sem o serviço de notificações, o resto continua

> Real, em produção e pequeno o bastante para caber na cabeça: cada requisito reaparece nos diagramas.

📖 Exemplo: projeto Vez — docs/01-visao-geral.md (§2, §4, §6) e docs/03-acesso-rbac.md

MOTIVAÇÃO

Três modos de usar UML

Esboço

É o nosso caso. Desenho seletivo e informal, feito para **conversar**. Mostra só o que importa naquela discussão.

Planta

Modelo detalhado e completo, para outra pessoa implementar sem ambiguidade. Times grandes, contratos.

Linguagem

O modelo é executável e gera o sistema. Abordagens de MDA — fora do escopo daqui.

Consequência prática: diagrama **incompleto não é defeito**, desde que o que está lá esteja certo e responda a alguma pergunta. O defeito é desenhar tudo sem saber por quê.

📖 FOWLER · *UML Essencial* · Cap. 1 (p. 25) e Cap. 2 (p. 39)

PANORAMA

14 diagramas — usamos cinco

Os diagramas da UML — e os cinco que vamos usar

A UML 2 define 14 diagramas. Ninguém usa todos. Fowler: use o que ajuda a pensar e comunicar; o resto é peso morto.

ESTRUTURAIS

o que o sistema É — as peças e como se ligam

Diagrama de Classes
Que conceitos existem no domínio e como se relacionam?

- Objetos
- Pacotes
- Componentes
- Instalação (deployment)
- Estruturas compostas
- Perfis

Os cinco em cinza existem e são úteis em outros contextos — não fazem parte desta disciplina.

No Milestone 2 e 3 você entrega:
classes + casos de uso + sequência + atividades/estados

COMPORTAMENTAIS

o que o sistema FAZ — ao longo do tempo

Diagrama de Casos de Uso
Quem usa o sistema, e para quê?

Diagrama de Sequência
Em que ordem os objetos conversam, para realizar um caso de uso?

Diagrama de Atividades
Qual é o fluxo do processo, com decisões e paralelismo?

Diagrama de Máquina de Estados
Por quais estados um objeto passa, e o que provoca a transição?

- Comunicação
- Temporização
- Visão geral da interação
- Casos de uso (detalhe)

Um diagrama existe para **responder a uma pergunta**. Se você não sabe qual pergunta ele responde, provavelmente não precisa dele.

FOWLER, UML Essencial, *cap. 1 · SOMMERVILLE 10ed · §5.2 a §5.4 · VALENTE · §4.2.2 Diagramas UML · OMG, UML 2.5.1*

📖 FOWLER · Cap. 1 · VALENTE · §4.2.2 · SOMMERVILLE 10ed · §5.1 a §5.4

AULA 28

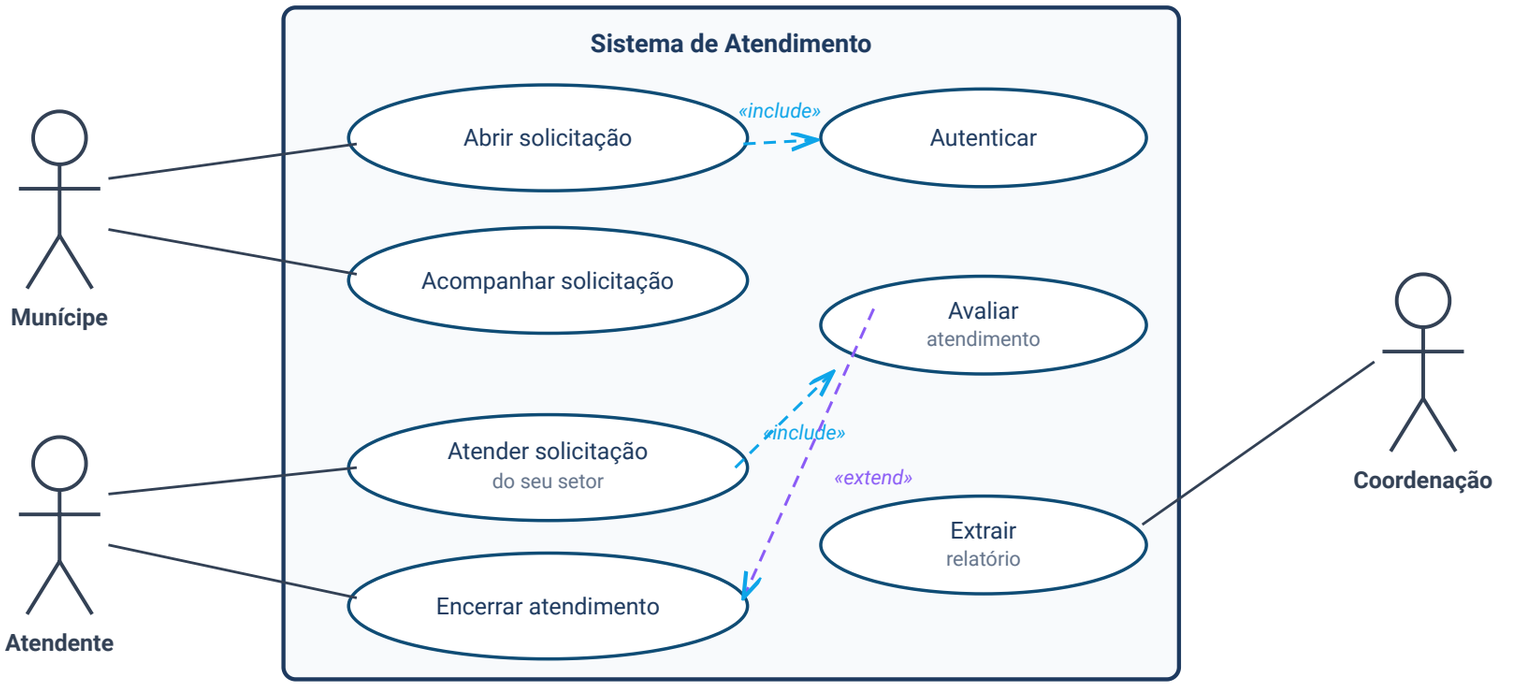
Casos de uso

NOTAÇÃO

Ator, caso de uso e fronteira

Diagrama de casos de uso — quem usa, e para quê

Ele não descreve fluxo nem tela: dá o **mapa de quem interage com o sistema** *e do que cada um consegue fazer.*



«include» — sempre acontece

O caso de uso incluído é parte obrigatória do que inclui.
Abrir solicitação **sempre** exige autenticar.

«extend» — só às vezes

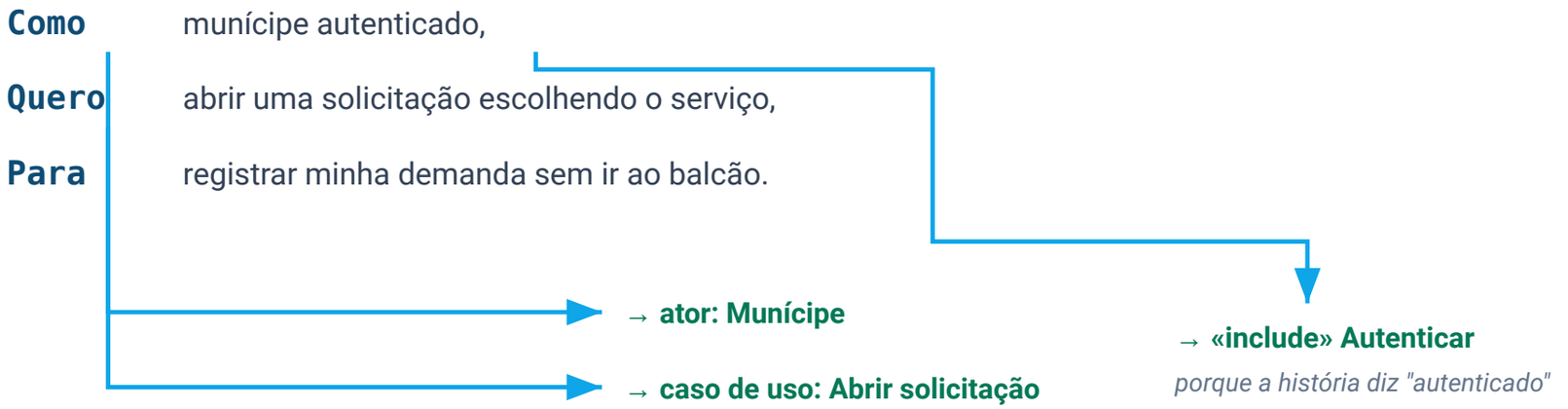
O caso de uso estendido acrescenta comportamento opcional.
Ao encerrar, o munícipe **pode** avaliar — ou não.

FOWLER, UML Essencial, *cap. 9 (p. 104) · SOMMERVILLE 10ed · §5.2.1 · VALENTE · §3.4 · PRESSMAN 9ed · §7.4*

📖 FOWLER · Cap. 9 (p. 104) · SOMMERVILLE 10ed · §5.2.1

PONTE

Da História de Usuário ao caso de uso



O "Para" não vira nada no diagrama — mas é o que permite questionar se o caso de uso deve existir.

RELAÇÕES

«include» × «extend»

«include» — sempre acontece

- O caso incluído é **parte obrigatória** do que o inclui.
- Serve para fatorar comportamento repetido.
- *Abrir solicitação **sempre** inclui autenticar.*

«extend» — só às vezes

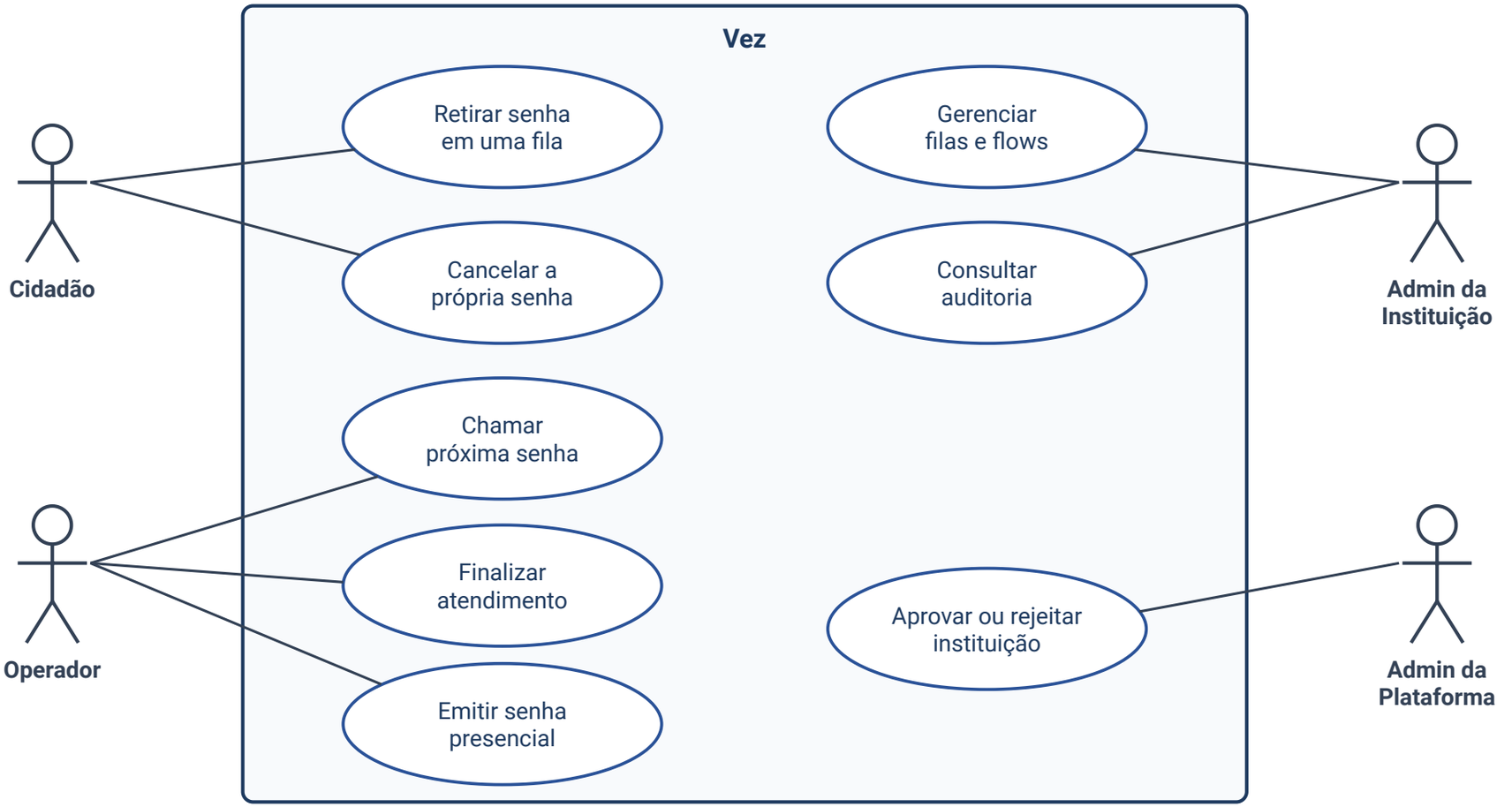
- Acrescenta comportamento **opcional**, sob condição.
- A seta aponta *para* o caso estendido.
- *Ao encerrar, o munícipe **pode** avaliar.*

Erro clássico: decompor casos de uso em passos. "Preencher formulário" e "Clicar em salvar" não são casos de uso — são passos de um. Caso de uso é um **objetivo completo**.

📖 FOWLER · Cap. 9 (p. 104)

CASO REAL: VEZ

Quem faz o quê: só o Operador chama senha



📖 Exemplo: projeto Vez — docs/06-api.md (diagrama de casos de uso) e docs/03-acesso-rbac.md. FOWLER · Cap. 9

PONTE

Cada história vira um caso de uso rastreável

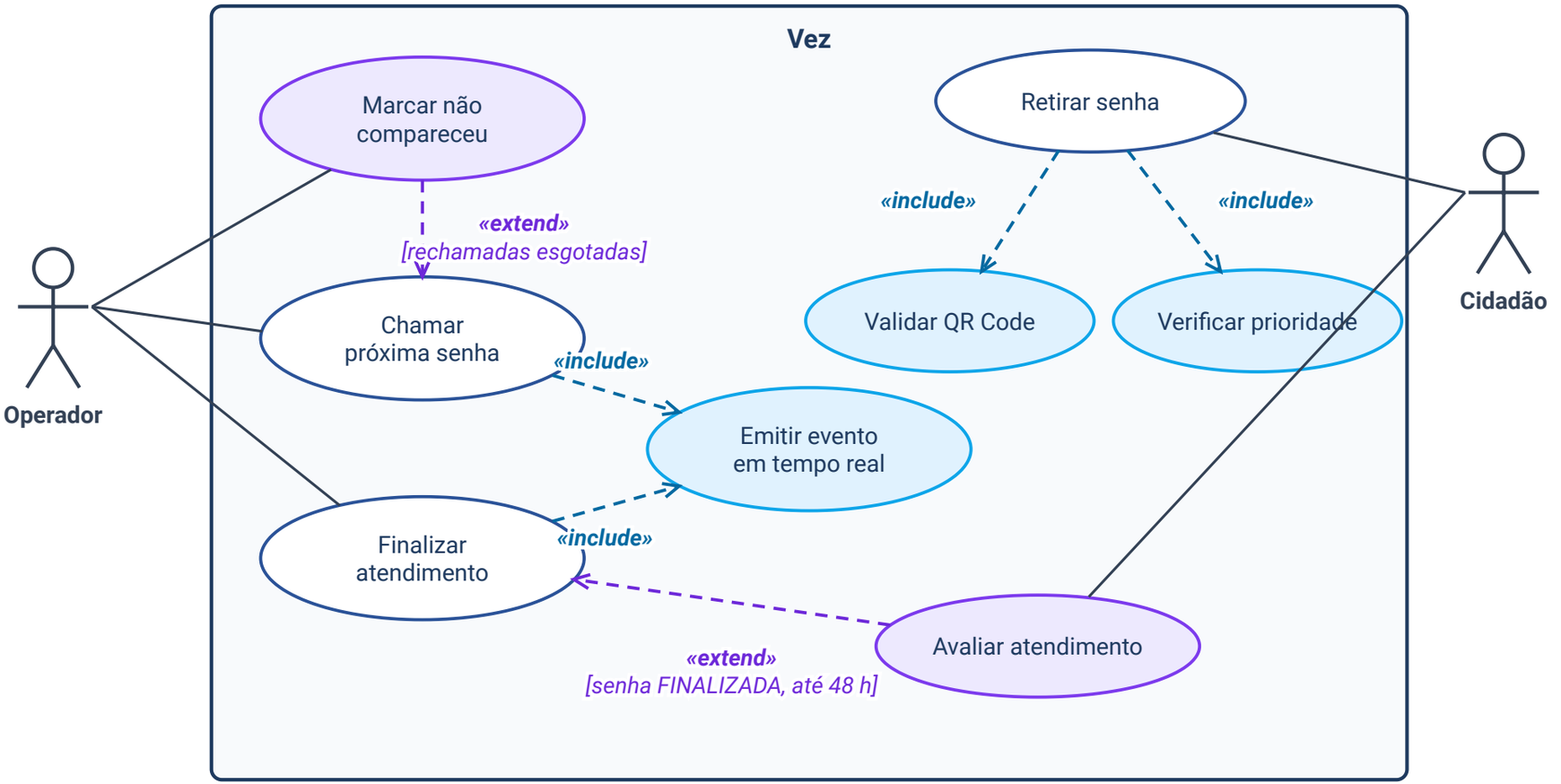
História (Como / Quero)	Ator	Caso de uso	Relação
US-V01 · retirar senha pelo celular	Cidadão	Retirar senha (UC06)	«include» Validar QR Code
US-V02 · chamar a próxima senha	Operador	Chamar próxima senha (UC12)	«include» Emitir evento
US-V03 · avaliar o atendimento	Cidadão	Avaliar atendimento (UC10)	«extend» Finalizar

Erro comum: o caso de uso virou **tela ou botão** ("Tela de fila", "Clicar em Chamar"). O caso de uso é o objetivo: *Chamar próxima senha*.

📖 Exemplo: projeto Vez — docs/06-api.md. US-V0x escritas para a aula.

CASO REAL: VEZ

include sai do caso-base; extend chega nele

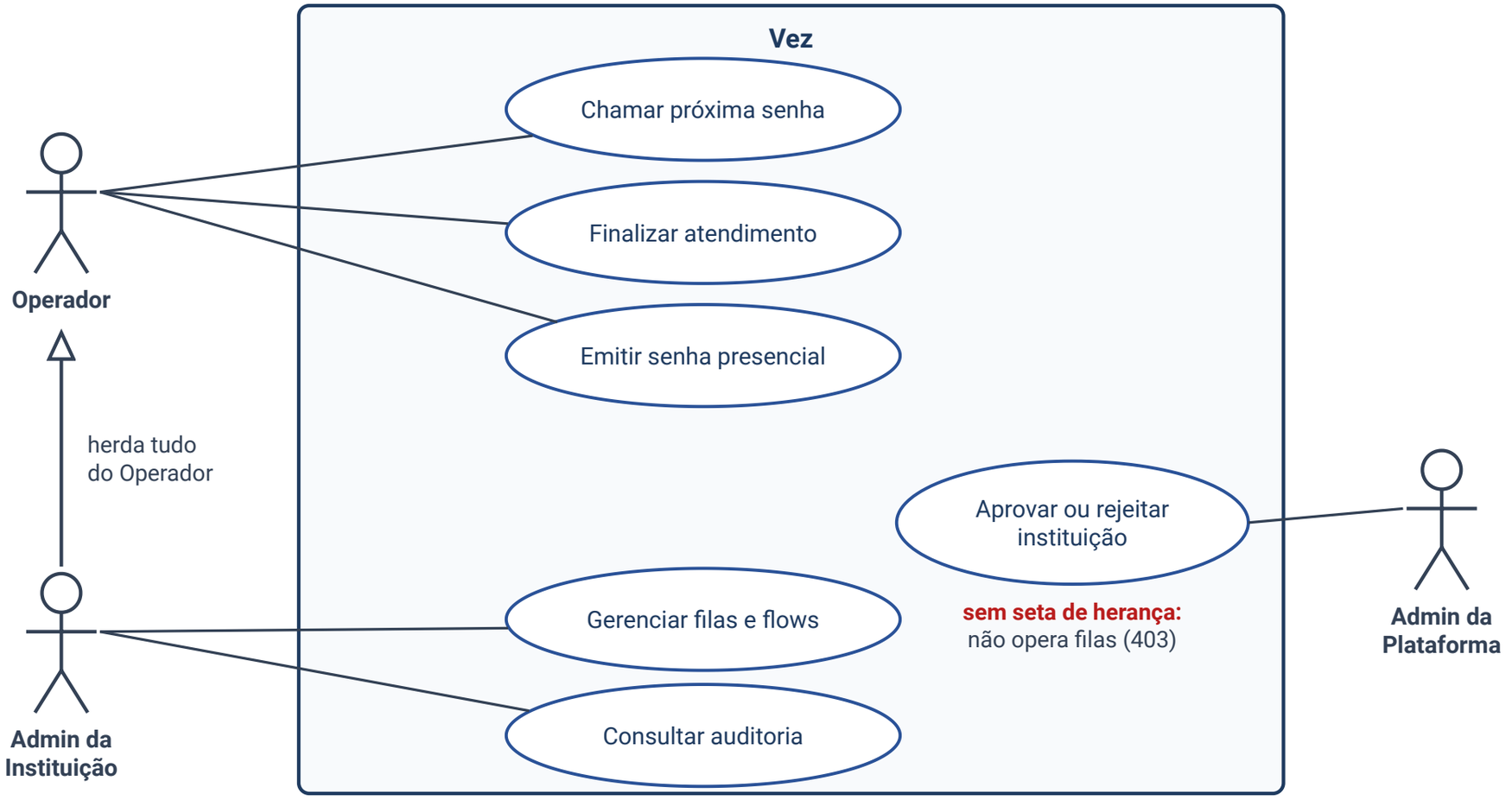


Seta tracejada: «include» sai do caso-base para o incluído; «extend» sai de quem estende para o caso-base.

📖 Exemplo: projeto Vez — docs/06-api.md. FOWLER · Cap. 9 (p. 104)

ATORES

Generalização: só se o ator herda tudo



📖 Exemplo: projeto Vez — docs/03-acesso-rbac.md e matriz de permissões em docs/06-api.md. FOWLER · Cap. 9

AULAS 29 A 31

Classes

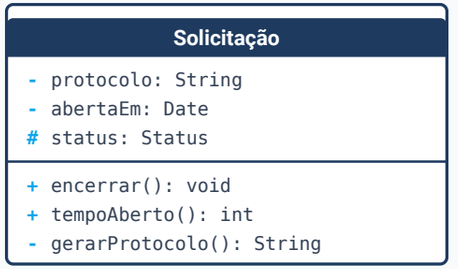
NOTAÇÃO

Anatomia e relacionamentos

Diagrama de classes — os conceitos do domínio

*Modele o **domínio do problema**, não as tabelas do banco. Nome de classe é substantivo do vocabulário do cliente.*

Anatomia de uma classe



nome · atributos · operações, separados por linha

Visibilidade

+	**pública** — qualquer classe acessa
-	**privada** — só a própria classe
#	**protegida** — a classe e suas subclasses
~	**pacote** — classes do mesmo pacote

Multiplicidade

`1`	exatamente um
`0..1`	nenhum ou um
`*`	zero ou muitos
`1..*`	um ou muitos
`2..5`	entre dois e cinco

Leia sempre nos dois sentidos.

Os cinco relacionamentos

Associação



Vínculo estrutural. Um munícipe abre várias solicitações.

Herança (generalização)



Triângulo vazado apontando para a superclasse. Atendente **é um** Usuário.

Agregação



Losango **vazado**: o todo agrupa as partes, mas elas **existem sem ele**.

Composição



Losango **preenchido**: apagou a solicitação, **apagou as mensagens**.

Dependência



Tracejada: uma classe **usa** a outra sem guardá-la.

FOWLER, UML Essencial, cap. 3 (p. 52) e cap. 5 (p. 77) · SOMMERVILLE 10ed · §5.3.1, §5.3.2 e §5.3.3 · VALENTE · §4.3

📖 FOWLER · Cap. 3 (p. 52) e Cap. 5 (p. 77) · SOMMERVILLE 10ed · §5.3

ALERTA

Domínio, não banco de dados

× Modelo de tabelas disfarçado

- Todos os atributos **públicos**, com get/set para cada um.
- `idProfessor: int` em vez de **associação** com Professor.
- Nomes como `tb_reserva`, `ReservaDTO`.
- Operações como `salvarNoBanco()`.

✓ Modelo de domínio

- Atributos **privados**; a classe tem comportamento próprio.
- Relacionamento desenhado como **linha**, com multiplicidade.
- Nomes do **vocabulário do cliente**.
- Operações que **fazem sentido no domínio**.

Teste rápido: **o cliente reconheceria o nome da classe?** Se não, provavelmente não é classe de domínio.

MULTIPLICIDADE

Leia sempre nos dois sentidos



→ Um munícipe abre **muitas** solicitações.

← Cada solicitação pertence a **um** munícipe.

A segunda leitura é a que revela erro — é ela que faz perguntar:

"duas pessoas podem compartilhar a mesma solicitação?"

📖 FOWLER · Cap. 3 (p. 52) · SOMMERVILLE 10ed · §5.3.1

DO DOMÍNIO AO PROJETO

Classe de domínio é do cliente; classe de projeto é da equipe

Domínio

Fila, Senha, Instituição. **O cliente reconhece o nome.** Serve para conversar sobre o problema.

Projeto

FilaService, FilaRepository, CustomError. **Existe para organizar o código.** Serve para conversar com a equipe.

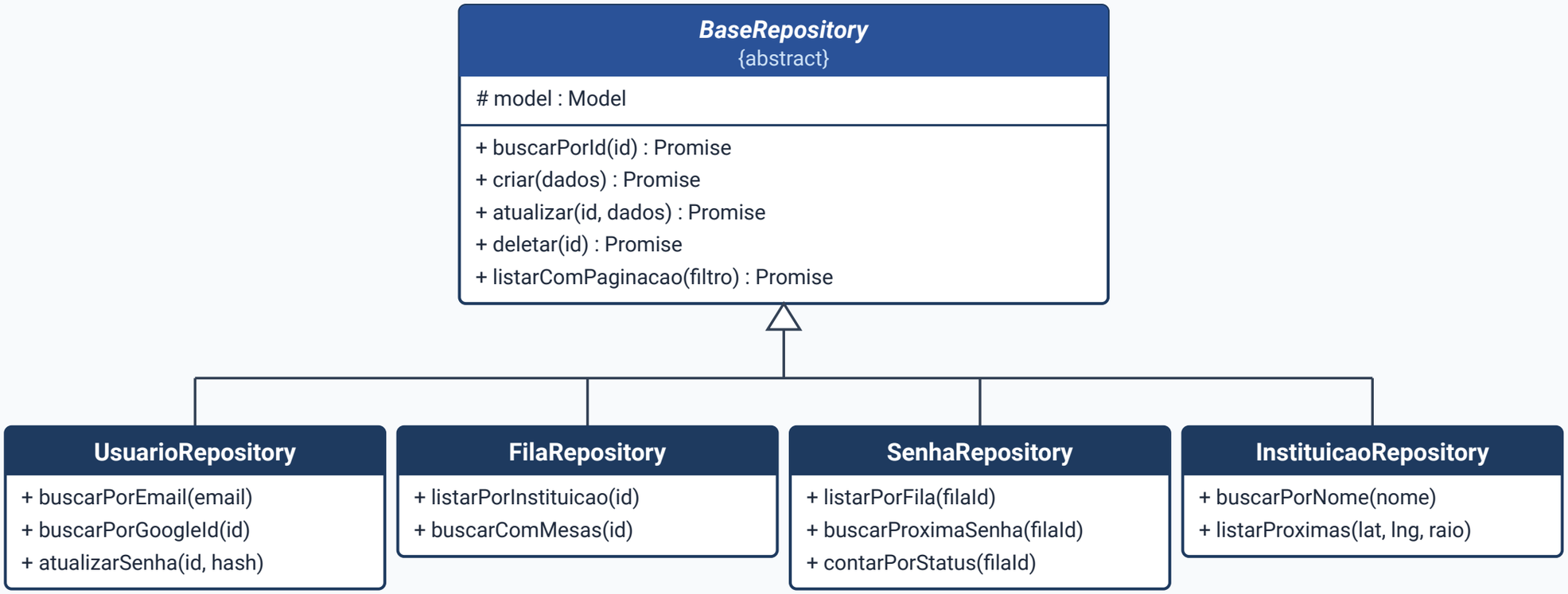
Erro comum: desenhar **todas** as classes do código. Escolha só as que respondem à pergunta do diagrama.

📖 Exemplo: projeto Vez · docs/06-api.md (Apêndice D) · FOWLER · Cap. 3 e 5

HERANÇA

O que é comum sobe para a classe abstrata

Repositórios do Vez: o que é comum fica no pai, o que é específico fica no filho



Herdado: os quatro já têm buscarPorId, criar, atualizar, deletar e listarComPaginacao, sem repetir.

Específico: só o que o negócio daquela entidade pede. {abstract} (itálico): ninguém faz «new BaseRepository»; só os filhos existem.

📖 Exemplo: projeto Vez — docs/diagramas/api-uml-2.svg · FOWLER · Cap. 5 (p. 77)

DEPENDÊNCIA

Entre camadas, uma classe usa a outra: tracejada, não losango

Quem conhece quem: a seta aponta para a classe que é usada



✓ Dependência (usada no Vez)

tracejada, ponta aberta: «A usa B».

O Controller recebe o Service pronto, no construtor (injeção de dependência); não o cria nem o apaga.

× Composição (não é o caso)

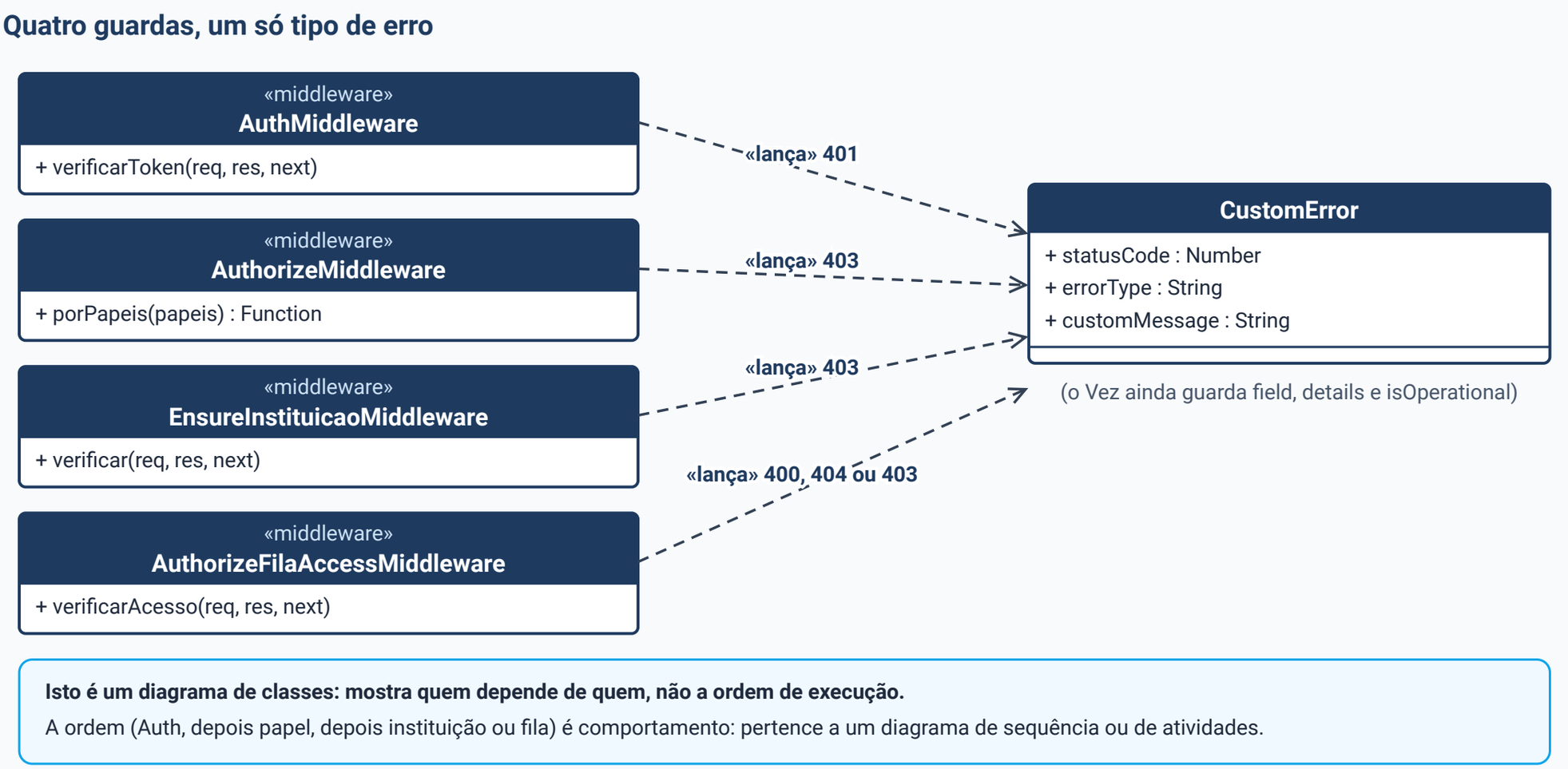
losango cheio: o todo é dono das partes.

Composição diria: apagou o Controller, apagou o Service. Mas FilaService existe sem ele, e o mesmo AuditoriaService atende vários serviços.

📖 Exemplo: projeto Vez — docs/diagramas/api-uml-3.svg e api-uml-4.svg · SOMMERVILLE 10ed · §5.3

DEPENDÊNCIA DE ERRO

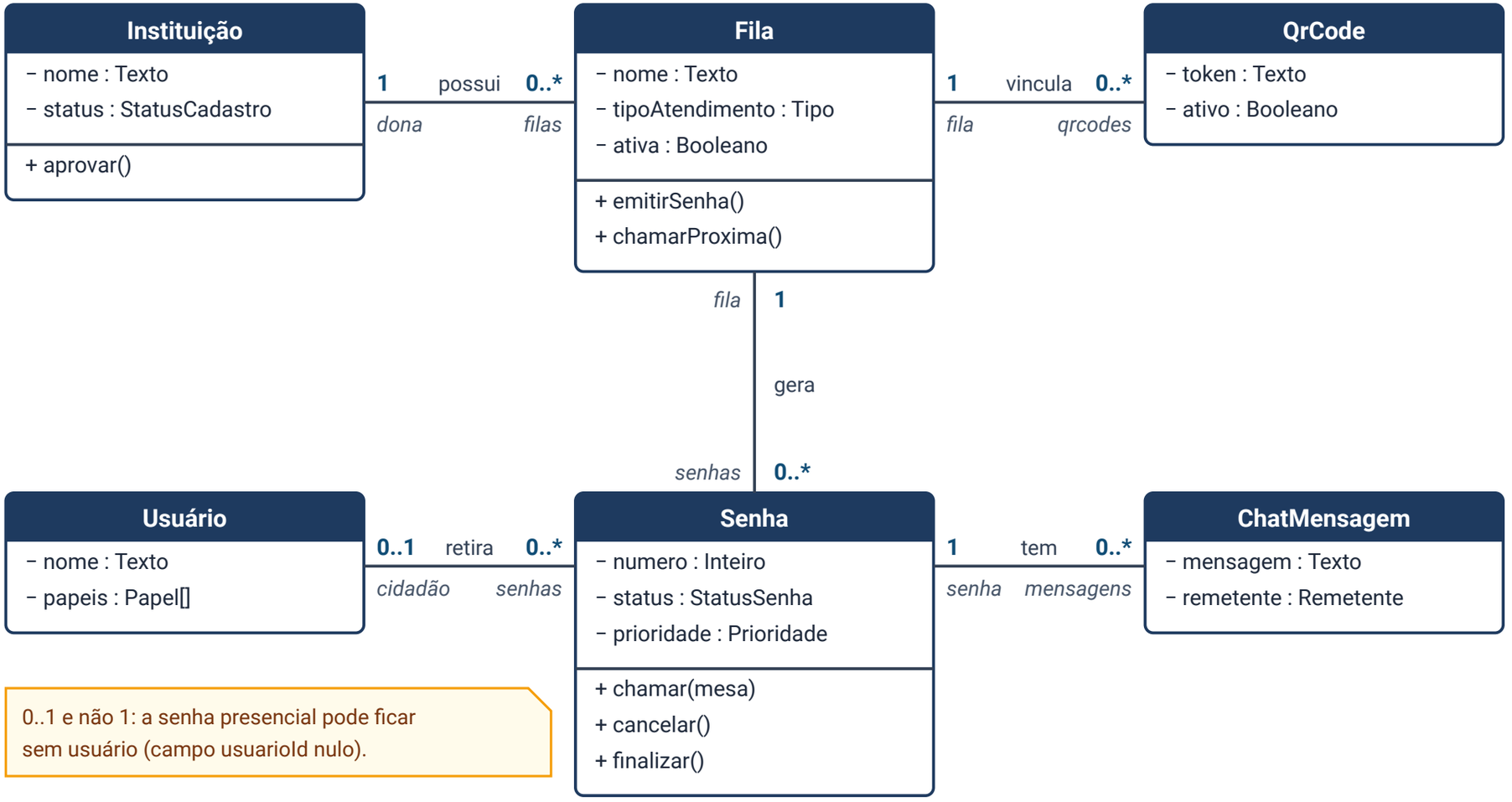
Quatro guardas dependem de um único CustomError



📖 Exemplo: projeto Vez — docs/diagramas/api-uml-5.svg e docs/03-acesso-rbac.md

EXEMPLO REAL: PROJETO VEZ

Seis classes do Vez já contam o negócio



📖 Exemplo: projeto Vez — docs/diagramas/api-uml-1.svg e docs/06-api.md (§7 e D.1) · FOWLER · Cap. 3 (p. 52)

MULTIPLICIDADE

O "1" do diagrama escondia um "0..1"

Lida num sentido só

"Um Usuário retira *muitas* Senhas." Parece certo, e o diagrama original do Vez parou aí, com `1` na ponta do Usuário.

Lida também no outro

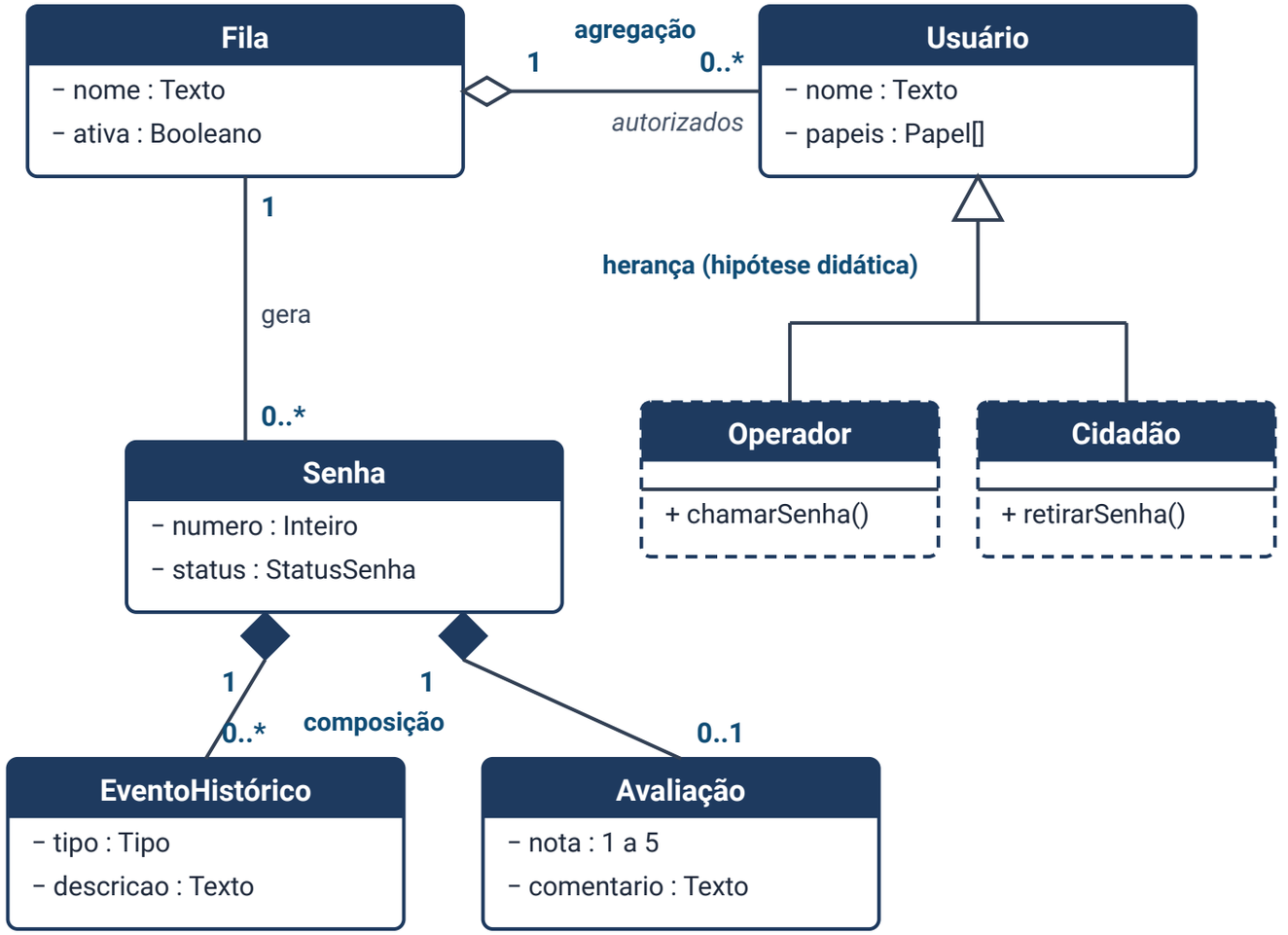
"Cada Senha tem *quantos* Usuários?" No Vez, a senha **presencial** pode não ter usuário (`usuarioId` nulo). Logo: `0..1` .

> A pergunta de negócio **"toda senha tem dono?"** decide o número. O que a tabela permite ou o que parece óbvio não decide.

📖 Exemplo: projeto Vez — docs/06-api.md §7.4 (Senhas) · FOWLER · Cap. 3 (p. 52)

RELACIONAMENTOS

Composição some com o todo; agregação, não



Como distinguir

Agregação

A parte existe sem o todo. Apagar a Fila não apaga os Usuários que a operam.

Composição

Apagou o todo, apagou a parte. Sem a Senha, o histórico e a avaliação não existem.

Herança

A subclasse é um tipo do pai.

No Vez real, papeis é um atributo: não há subclasses de Usuário.

📖 Exemplo: projeto Vez — api-fila/src/models (Fila, Senha) e docs/06-api.md · SOMMERVILLE 10ed · §5.3.2 e §5.3.3

MODELE O DOMÍNIO

A Senha no banco tem campos que o cliente nunca citou

Domínio: classe Senha



Associações: Fila [1] · Usuário [0..1]

Regra: o cidadão só cancela senha com status AGUARDANDO.

Regra: um operador atende uma senha por vez.

Banco: coleção senhas (MongoDB)

`_id : ObjectId`	fora: chave do banco
`filaId : ObjectId`	vira associação com Fila
`usuarioId : ObjectId | null`	vira associação 0..1 com Usuário
`instituicaoId : ObjectId`	fora: repete a Instituição da Fila
`numero · status · prioridade`	fica: atributos do domínio
`idClienteOffline : String`	fora: sincronização do totem
`createdAt · updatedAt`	fora: carimbos automáticos
`índices (filaId, status…)`	fora: desempenho de consulta

Pergunta de ouro: o cliente reconheceria isso? Fila, Usuário e "cancelar a senha" sim; _id e createdAt não.

📖 Exemplo: projeto Vez — docs/06-api.md §7.4 e api-fila/src/models/Senha.js · VALENTE · §4.3

AULA 32

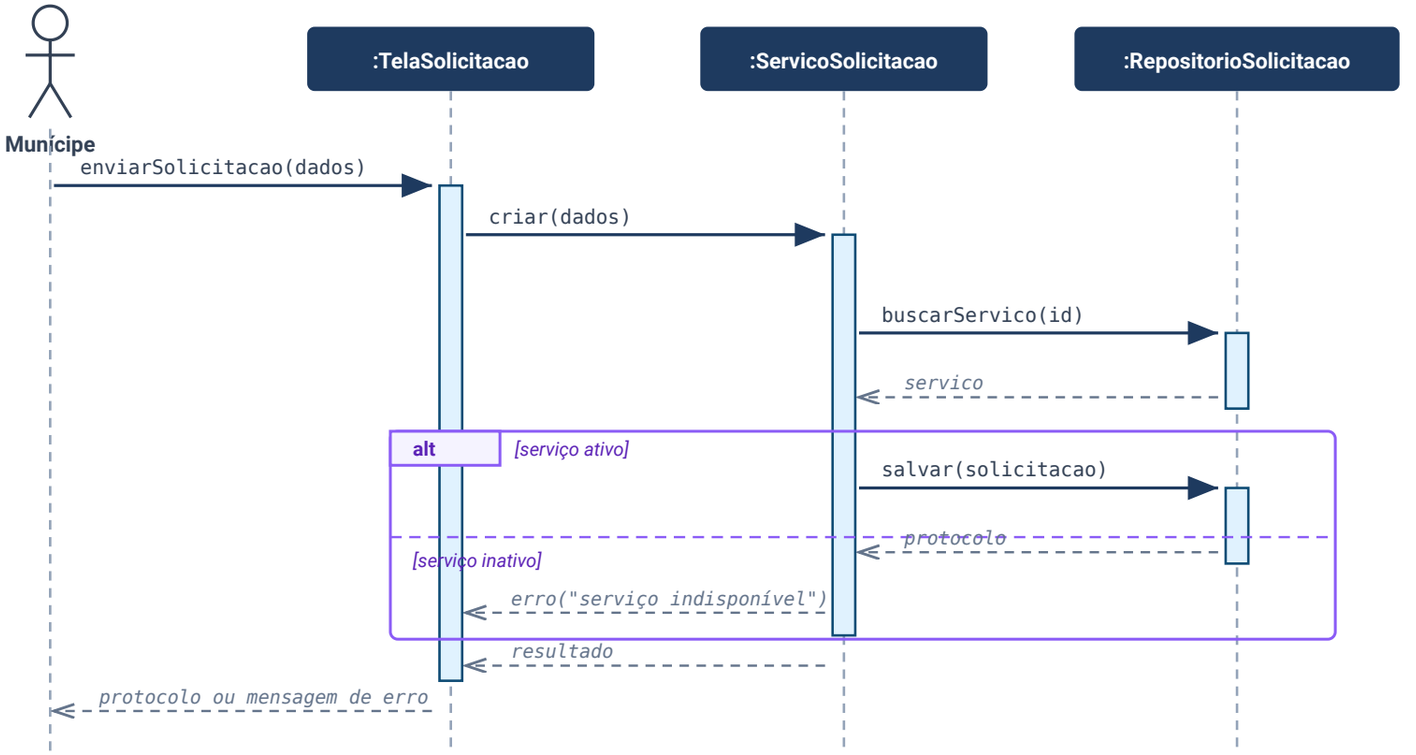
Sequência

NOTAÇÃO

A ordem da conversa

Diagrama de sequência — a ordem da conversa

Mostra **um cenário***: quais objetos trocam mensagens, em que ordem, para realizar um caso de uso. O tempo corre de cima para baixo.*



Barra de ativação

O retângulo na linha de vida marca **quando o objeto está executando**.

Fragmento alt

Caminhos alternativos: a **guarda** entre colchetes decide qual ocorre.

FOWLER, UML Essencial*, cap. 4 (p. 67) · SOMMERVILLE 10ed · §5.2.2 · VALENTE · §4.5*

📖 FOWLER · Cap. 4 (p. 67) · SOMMERVILLE 10ed · §5.2.2 · VALENTE · §4.5

PONTE

Um cenário Gherkin = um diagrama

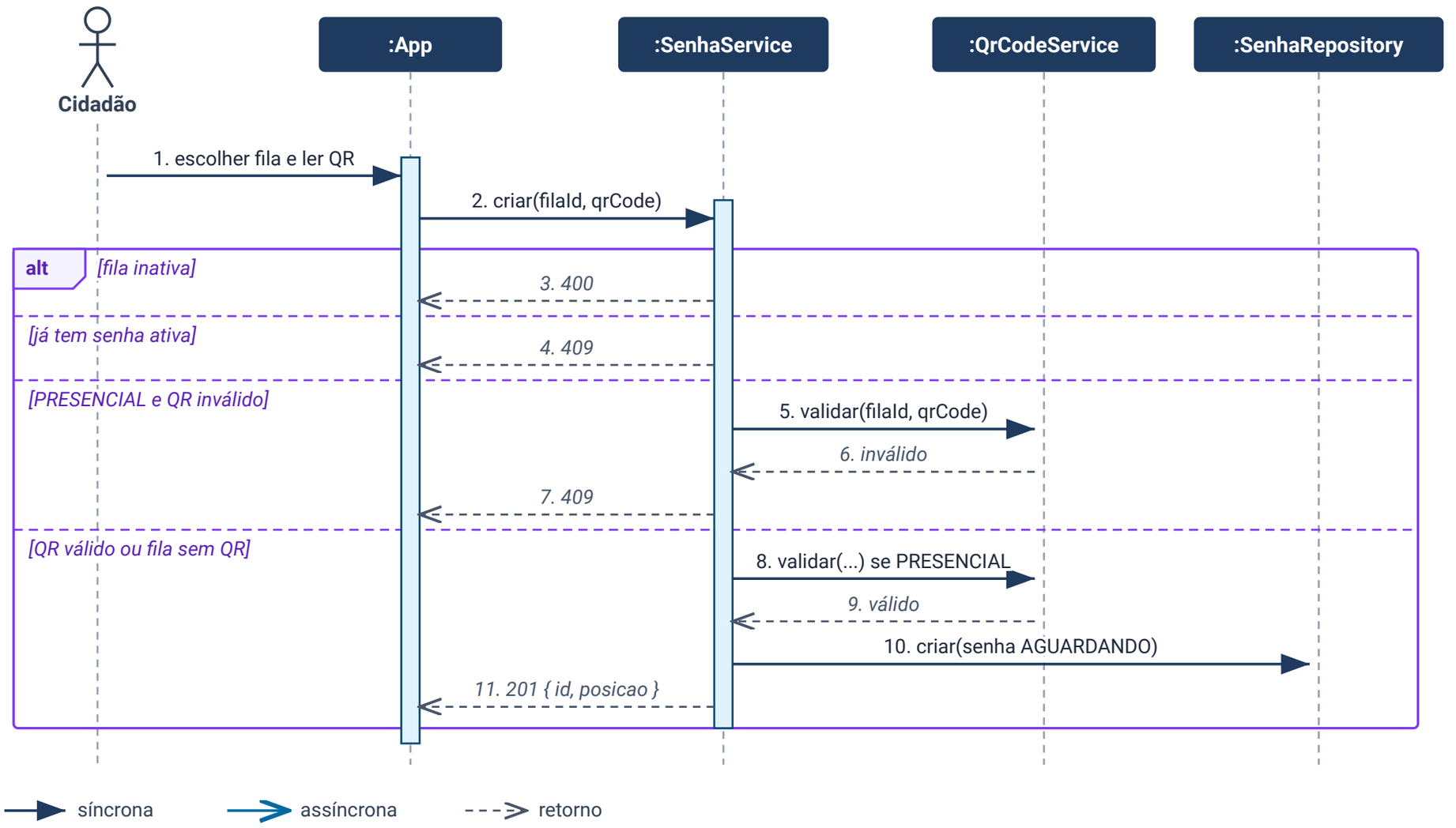
Dado	que estou autenticado e escolhi um serviço	→	**estado inicial dos participantes**
Quando	envio a solicitação	→	**a mensagem que inicia a sequência**
Então	recebo um protocolo	→	**o retorno ao final do diagrama**

O cenário de exceção vira o outro ramo do fragmento alt.

Não desenhe sequência para tudo. Vale a pena para cenários com colaboração interessante. Escolha os **dois ou três cenários principais** do projeto.

EXEMPLO: PROJETO VEZ · 1 DE 4

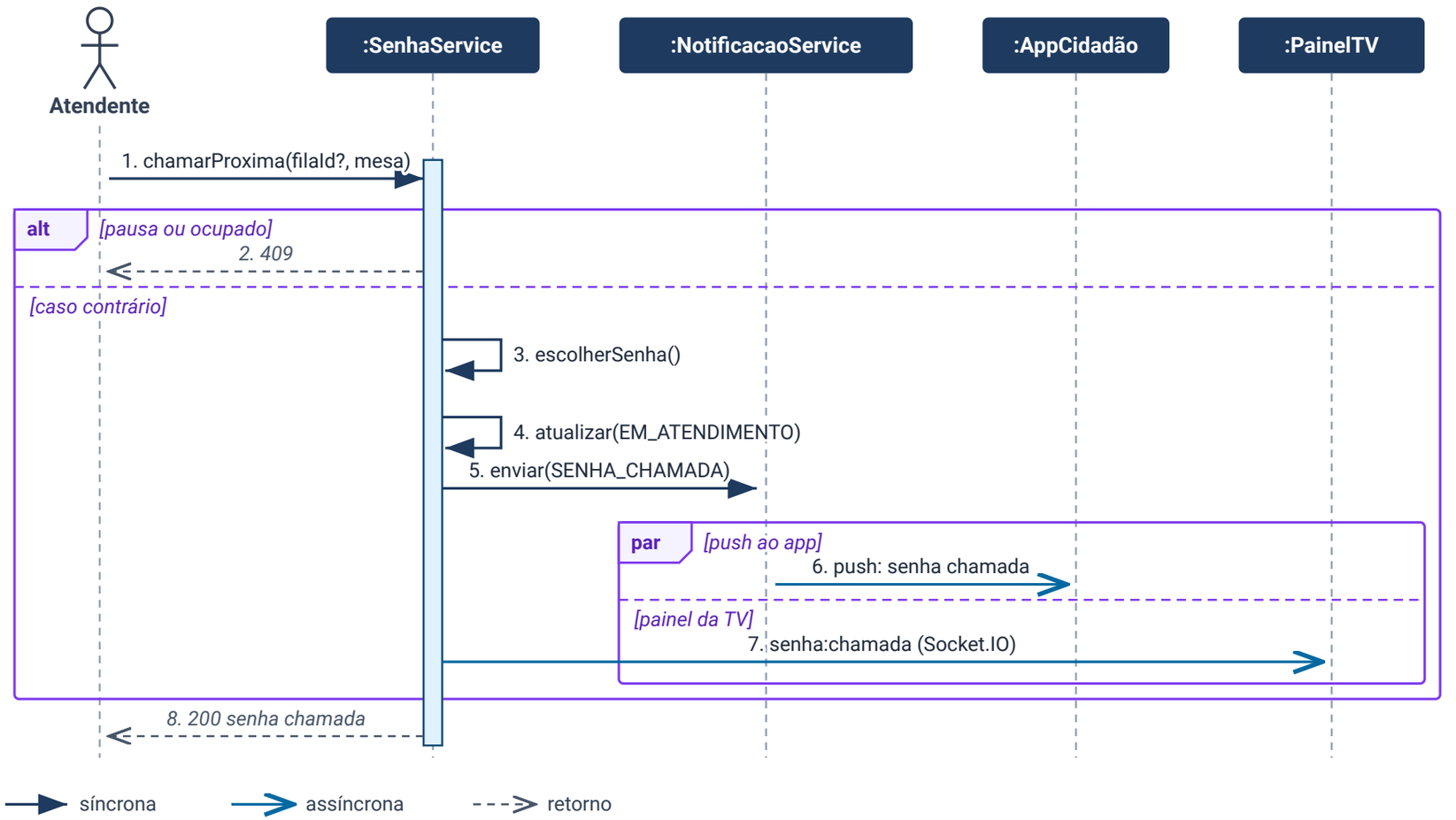
Retirar senha: o `alt` guarda as recusas



📖 Exemplo: projeto Vez — docs/06-api.md §8.1 (regras de criação) e §8.2 (QR Code); docs/diagramas/api-uml-10.svg · FOWLER · Cap. 4 (p. 67)

EXEMPLO: PROJETO VEZ · 2 DE 4

Chamar a próxima: avisos assíncronos em paralelo

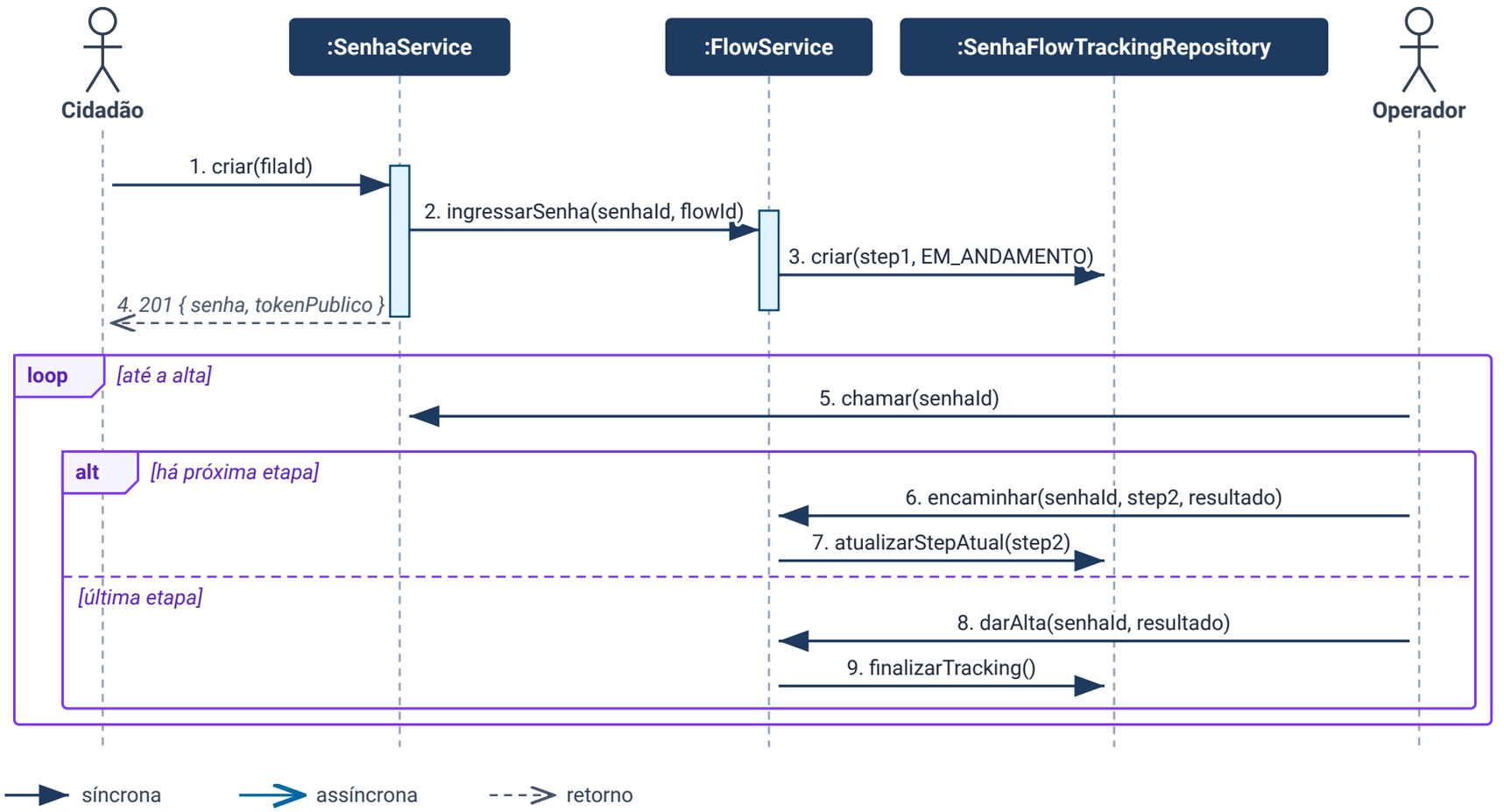


Assíncrona (6, 7): dispare e siga. **Self-message** (3, 4): a seta volta à própria linha de vida.

📖 Exemplo: projeto Vez — docs/25-operacao-atendimento.md ("Chamar a próxima: o fluxo único"); docs/08-notificacoes.md; docs/diagramas/api-uml-10.svg

EXEMPLO: PROJETO VEZ · 3 DE 4

Várias etapas: `loop` e `alt`

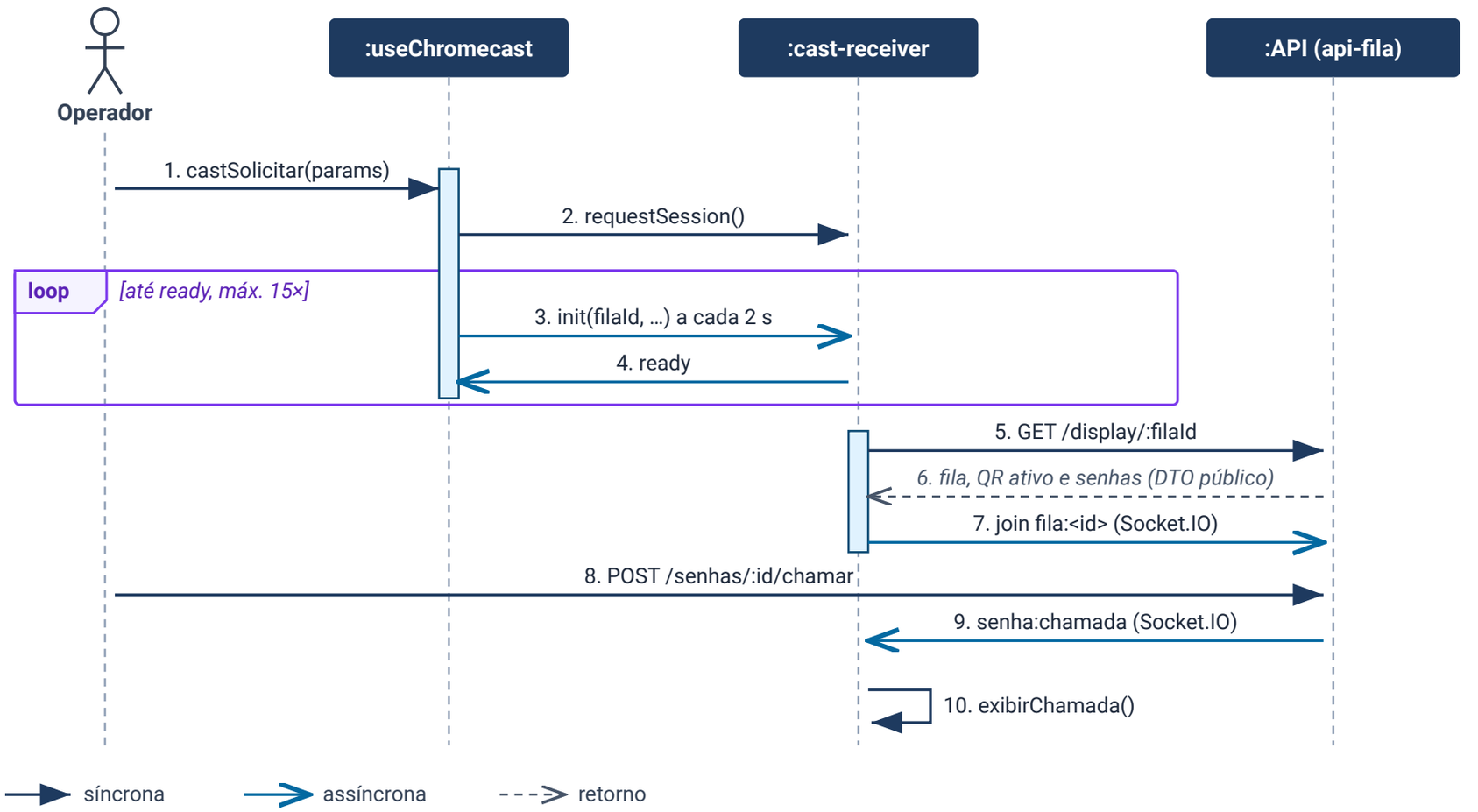


> O `loop` repete **até a alta**; o `alt` decide: **encaminhar** à próxima etapa ou **dar alta**. (UPA: Triagem → Consulta)

📖 Exemplo: projeto Vez — docs/06-api.md (Flow de atendimento, §A e E.7); docs/diagramas/api-uml-12.svg

EXEMPLO: PROJETO VEZ · 4 DE 4

Sistemas também trocam mensagens



Participantes **componentes**, não classes de domínio. O `loop` repete `init` até a TV responder `ready`.

📖 Exemplo: projeto Vez — docs/10-projecao-tv.md (§3.5, §7); docs/diagramas/projecao-tv-2.svg

REGRA DE OURO

O diagrama de sequência valida a classe

✓ Toda seta é uma operação

A mensagem `chamarProxima(filaId, mesa)` obriga a classe **`SenhaService`** a ter essa operação. Se ela não existe no diagrama de classes, **um dos dois está errado**.

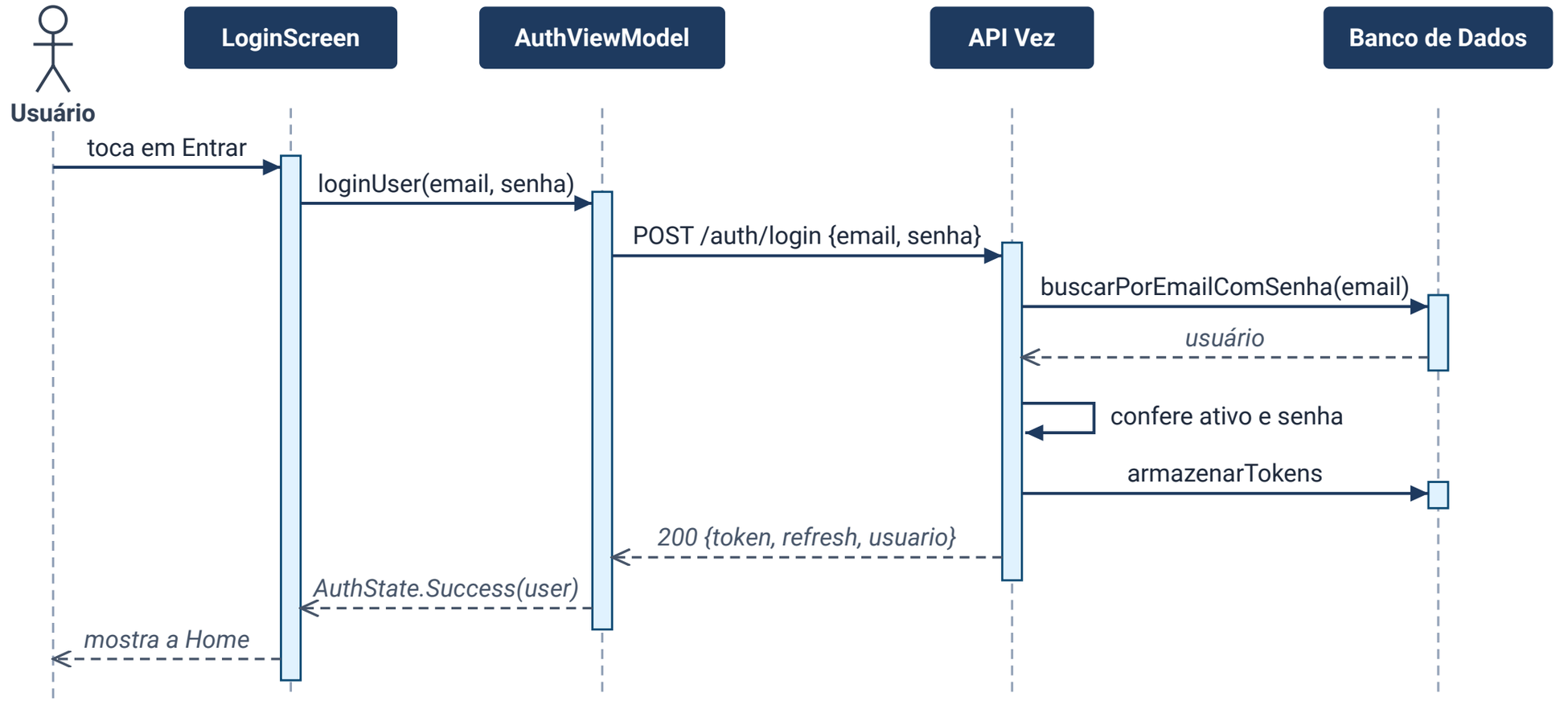
× Tela no meio, sem função

`:TelaFila → :TelaSenha: mostrar()`. Tela que só repassa e não decide nada não ajuda a validar classe nenhuma.

Mini-exercício. No diagrama de retirar senha, a mensagem 5 é `validar(...)`. De qual classe é essa operação?
Resposta: de `QrCodeService`, quem **recebe** a seta (e `SenhaService` passa a depender dela).

EXEMPLO REAL: VEZ

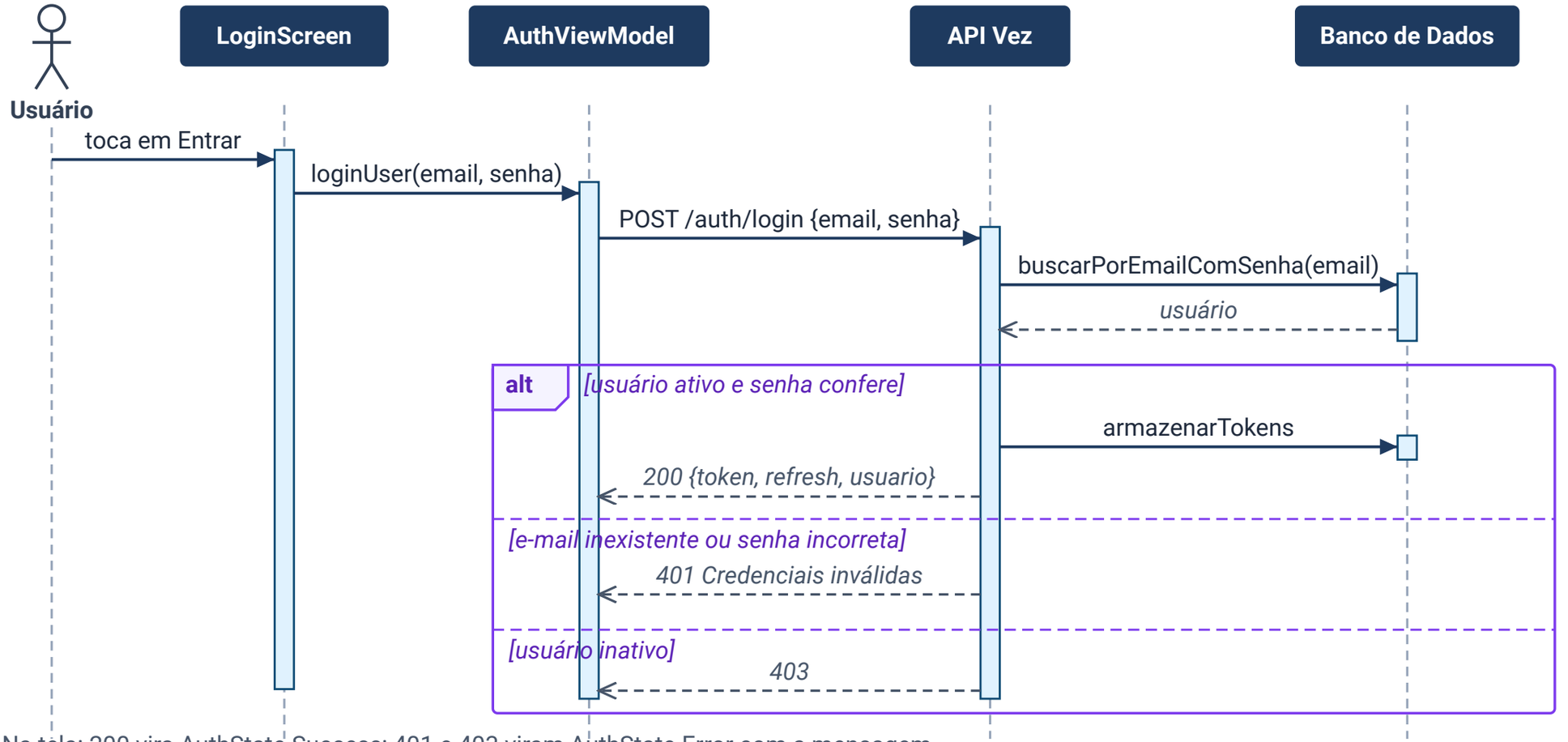
O login do Vez em uma conversa de cinco participantes



📖 Exemplo: projeto Vez, docs/13-mobile-v1a-auth-api.md e docs/06-api.md §9.1. FOWLER · Cap. 4 (p. 67)

FRAGMENTO ALT

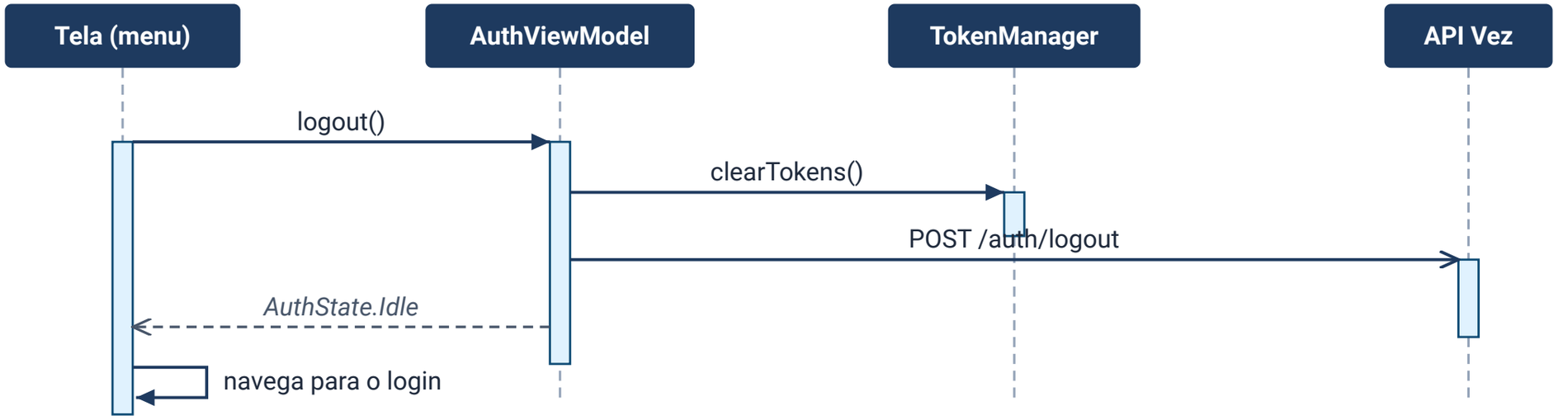
O `alt` põe o segundo cenário no mesmo diagrama



Erro comum: uma lifeline para cada classe do código. Mantenha **até 5 lifelines e ~12 mensagens**.

TIPOS DE SETA

Síncrona espera, assíncrona segue, retorno devolve



síncrona: quem chama espera

assíncrona: quem chama segue sem esperar

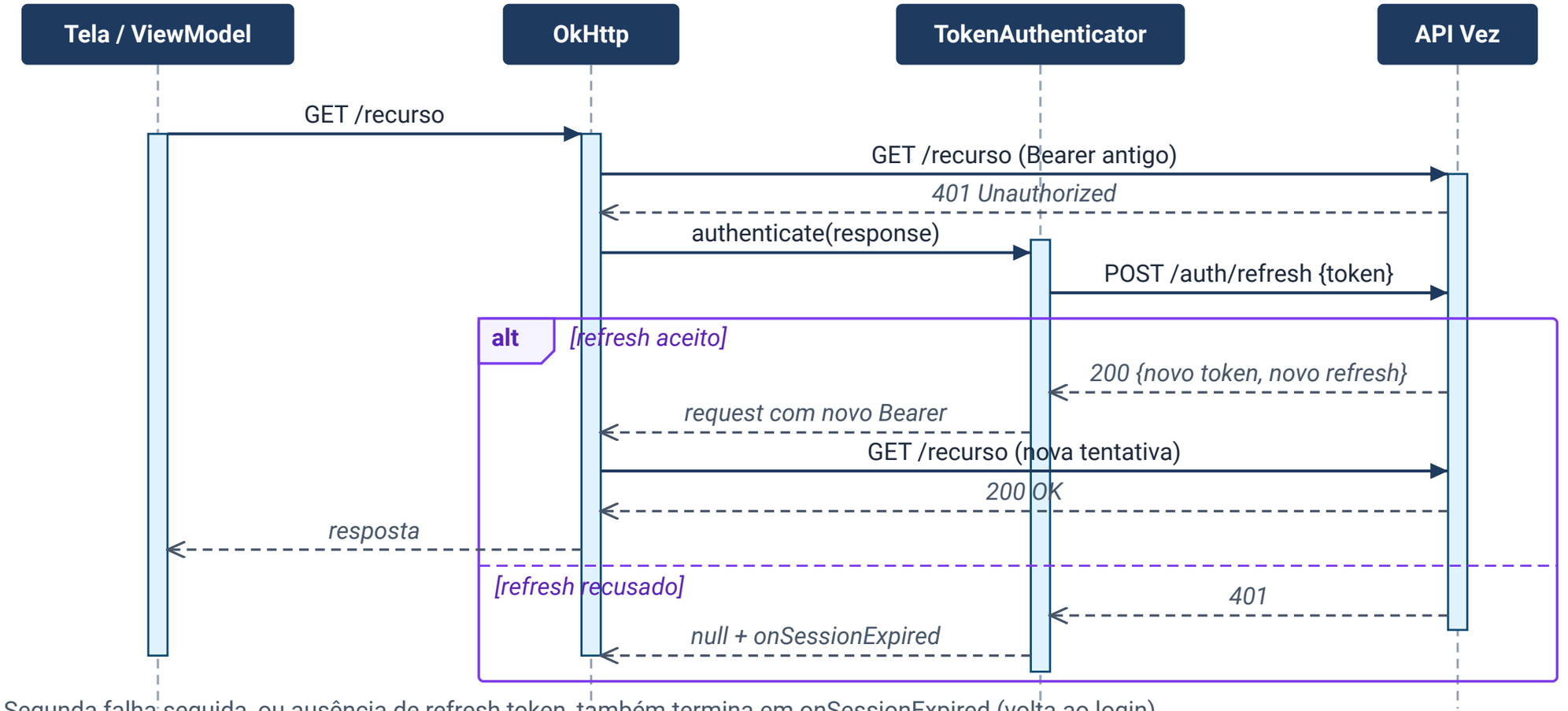
retorno: a resposta volta

API sem retorno desenhado: erros são ignorados.

📖 Exemplo: projeto Vez, diagrama mobile-v1a-auth-api-fluxo-7. FOWLER · Cap. 4 (p. 67)

RENOVAÇÃO DE SESSÃO

Token expirou? 401, refresh e nova tentativa, uma vez só

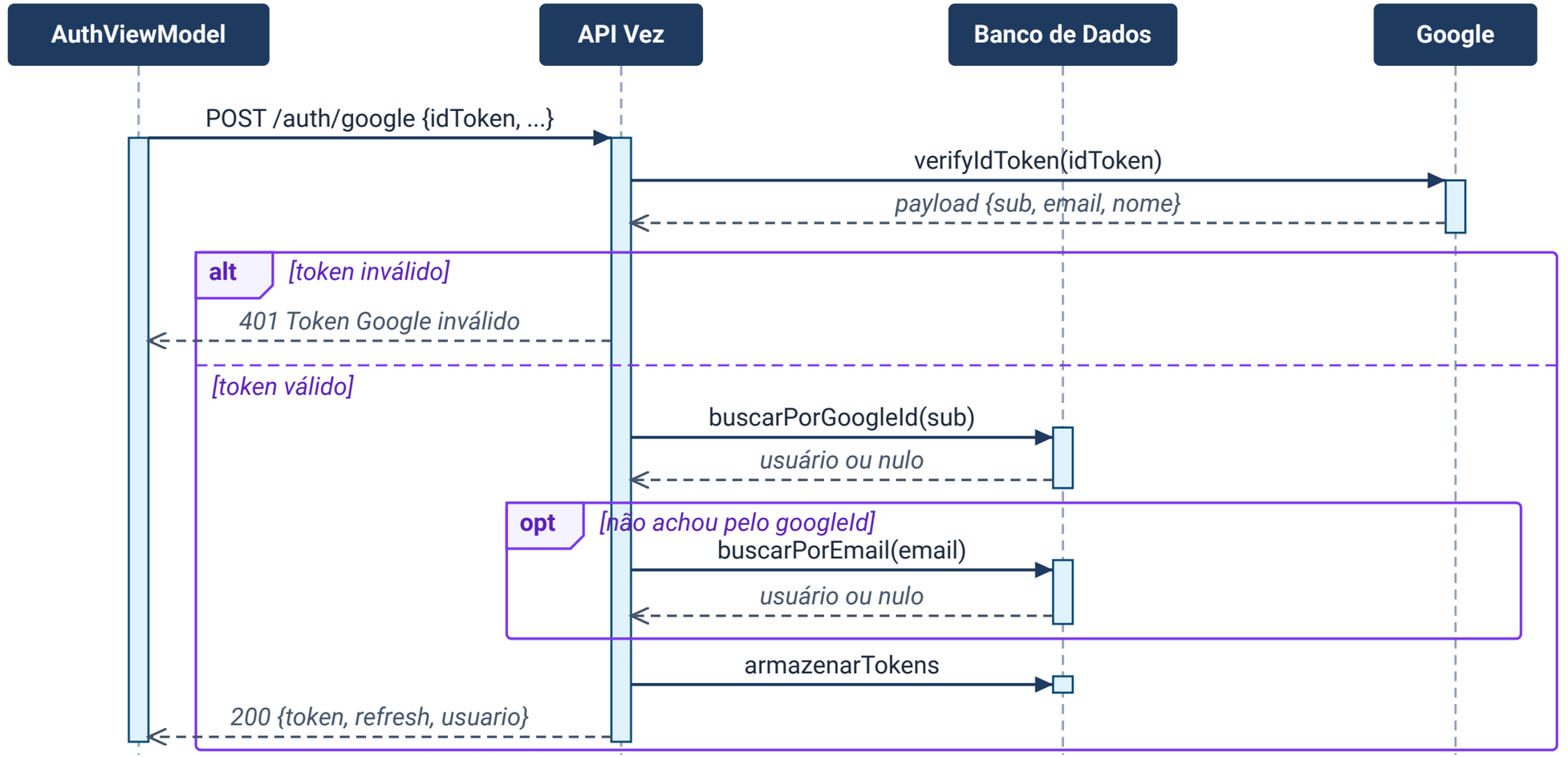


Segunda falha seguida, ou ausência de refresh token, também termina em onSessionExpired (volta ao login).

O Vez tenta **no máximo 1 refresh** por pedido: por isso é `alt`, não `loop`.

FRAGMENTO OPT

O `opt` guarda o que só acontece às vezes



📖 Exemplo: projeto Vez, docs/04-login-google.md (login Google, só no app Android). FOWLER · Cap. 4 (p. 67)

AULAS 33 E 34

Atividades e estados

COMPARAÇÃO

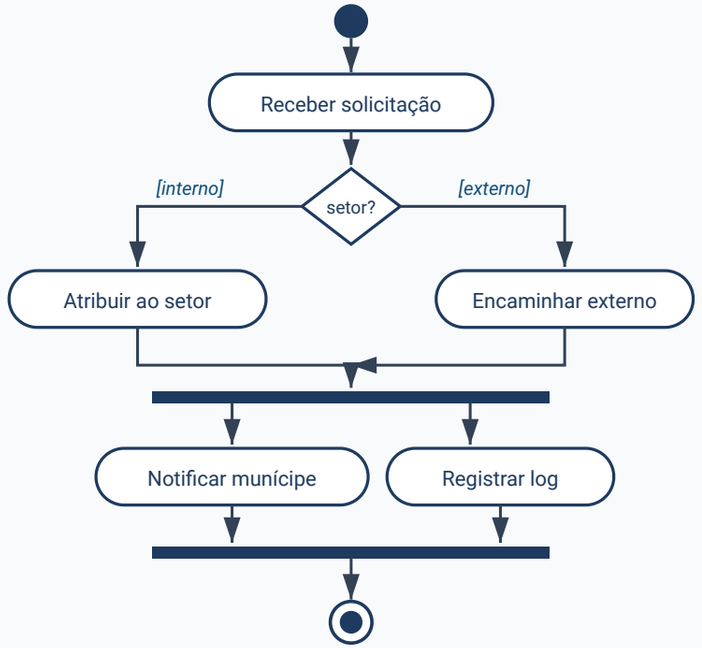
Dois olhares sobre o comportamento

Atividades × Estados — dois olhares sobre o comportamento

Atividades seguem **o processo**. *Estados seguem* **um objeto**. *Confundir os dois é o erro mais comum desta unidade.*

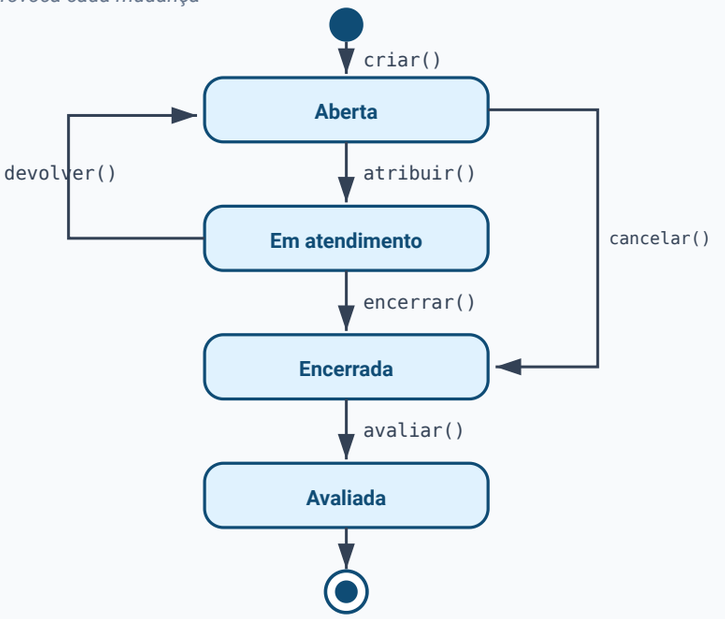
Diagrama de Atividades

Segue o fluxo do processo: quem faz o quê, em que ordem



Máquina de Estados

Segue um objeto: por quais estados passa e o que provoca cada mudança



Use atividades quando a pergunta for "como o trabalho flui?" — há decisões, paralelismo e vários papéis.

Use estados quando um objeto tiver ciclo de vida rico — e regras que dependem do estado atual.

FOWLER, UML Essencial, *cap. 11 atividades (p. 118) e cap. 10 máquina de estados (p. 110) · SOMMERVILLE 10ed · §5.4 · VALENTE · §4.6*

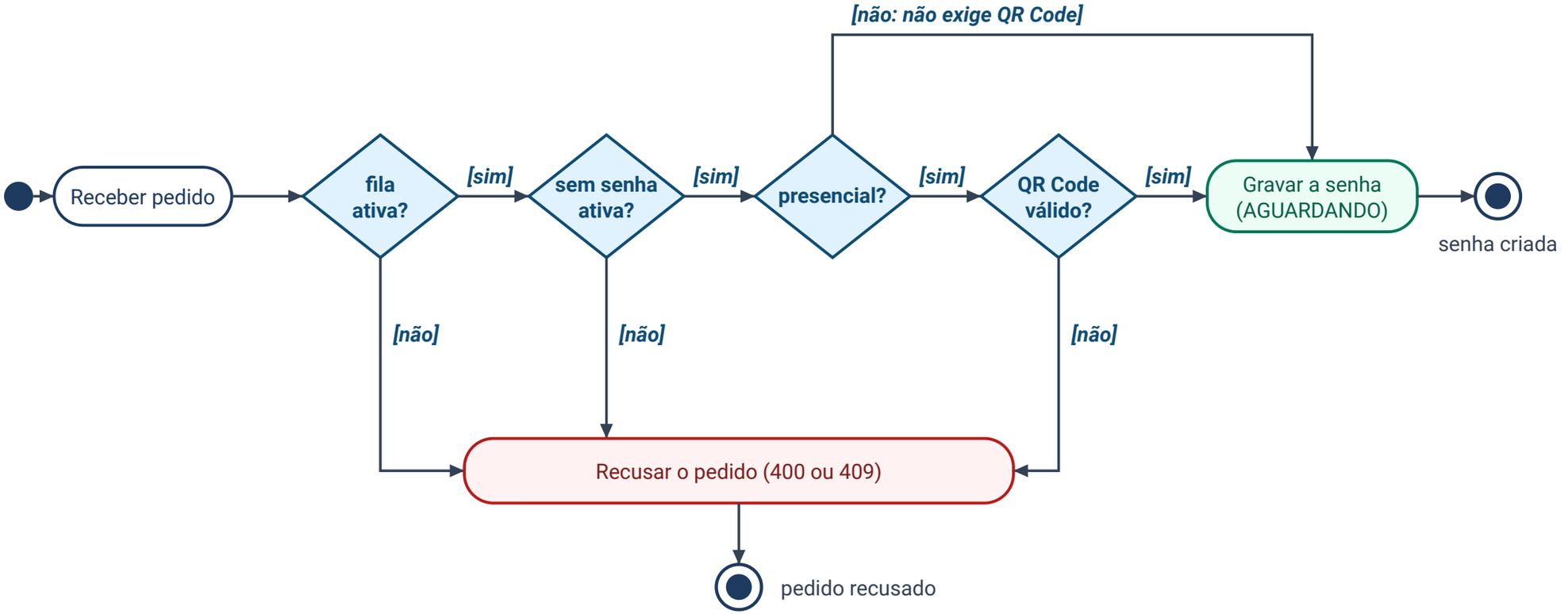
📖 FOWLER · Cap. 11 (p. 118) e Cap. 10 (p. 110) · SOMMERVILLE 10ed · §5.4

ATIVIDADES · EXEMPLO VEZ

Cada losango precisa de guardas que cobrem tudo

Criação de senha (POST /filas/{id}/senhas): o fluxo de decisões

Pergunta: em que ordem a API decide se a senha pode ser criada?



Cada decisão tem duas guardas que não se sobrepõem e cobrem todos os casos.

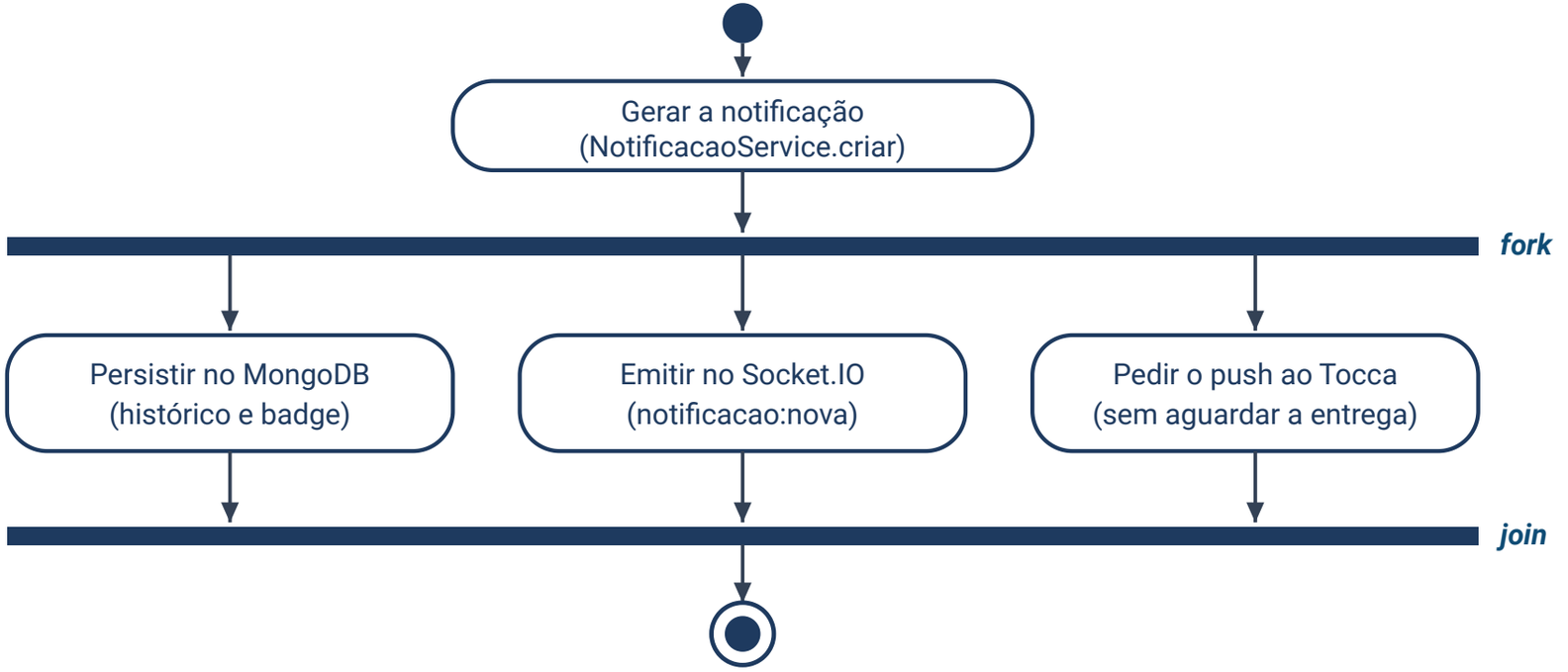
📖 Exemplo: projeto Vez, docs/06-api.md §8.1 e §8.2 · FOWLER · Cap. 11 (p. 118)

PARALELISMO

Fork abre caminhos; join espera todos

Notificação no Vez: três ações em paralelo

Pergunta: o que acontece ao mesmo tempo, e quando o fluxo continua?



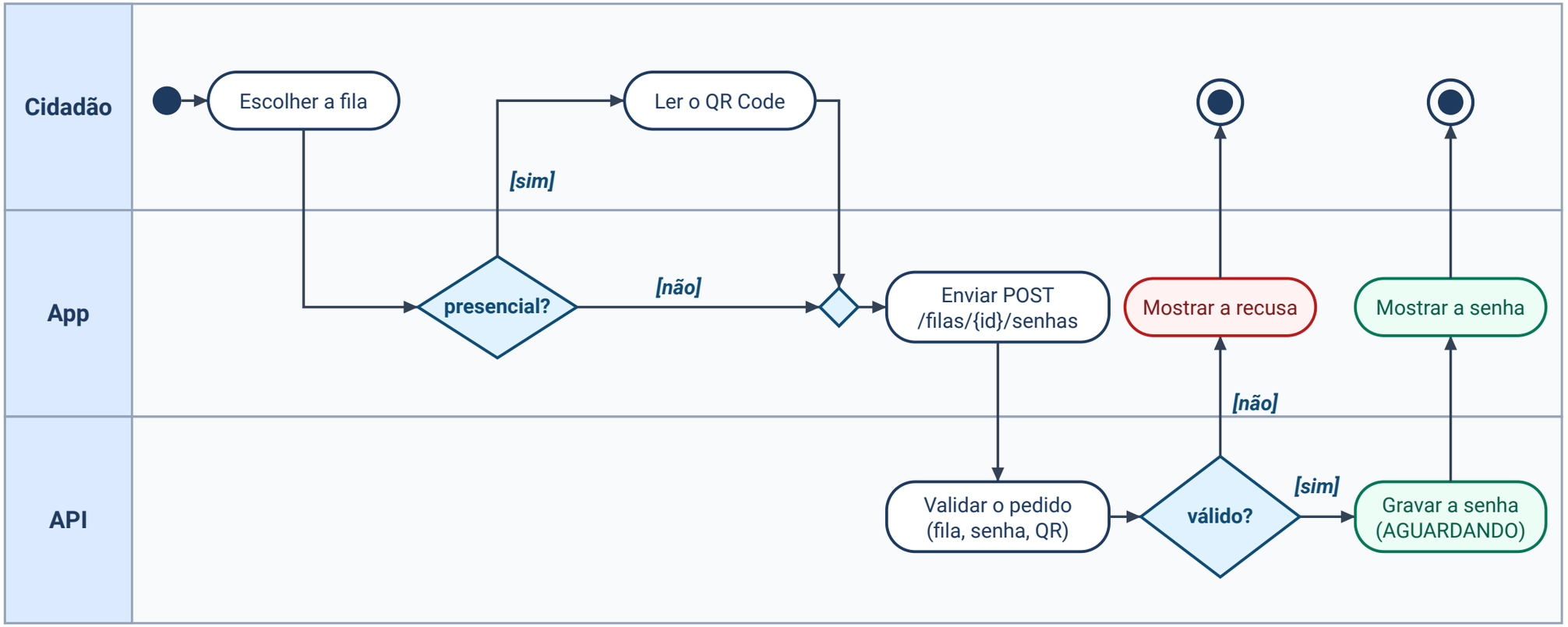
Fork: as três ações começam juntas, sem ordem entre elas.
Join: o fluxo só segue quando as três terminam (aqui, quando o pedido de push foi enviado).
Se o Tocca cair, as outras duas ações continuam.

📖 Exemplo: projeto Vez, docs/08-notificacoes.md · FOWLER · Cap. 11 (p. 118)

RESPONSÁVEIS

Partições mostram quem faz cada ação

Criação de senha com partições: quem faz o quê



A raia diz quem executa a ação. As setas que cruzam a linha entre raias são as trocas entre quem faz o quê.

📖 Exemplo: projeto Vez, docs/06-api.md §8.1 · FOWLER · Cap. 11 (p. 118)

NÃO CONFUNDA

Atividade mostra o fluxo; sequência mostra a conversa

Atividades

"Se a fila está ativa **e** o QR é válido, grava a senha."

Pergunta: quais passos e decisões, em que ordem?

Sequência

App → API: `POST /filas/{filaId}/senhas`; API → Fila: consultar; API → App: `201`.

Pergunta: quem envia qual mensagem a quem?

Ligação com Gherkin: cenário com várias condições (`Dado … E …`) vira decisões no diagrama de atividades; cada cenário é um caminho.

📖 Exemplo: projeto Vez, caso de uso Criar senha · FOWLER · Cap. 4 e Cap. 11

ALERTA

A confusão que derruba a nota

× Atividades com estados dentro

Caixas escritas:

"Aberta" → "Em atendimento" → "Encerrada"

Isso é máquina de estados desenhada com a notação errada.

✓ Atividades com ações

Caixas escritas:

"Receber solicitação" → "Atribuir ao setor" → "Notificar munícipe"

Verbo no infinitivo: alguém **faz** aquilo.

> **Teste:** ação é o que **alguém faz**; estado é como o objeto **fica**. Se cabe um verbo no infinitivo, é ação.

DECISÃO

Preciso de máquina de estados?

Procure no seu diagrama de classes um atributo chamado `status`, `situacao` ou `estado`.

Existe

Aquela classe tem **ciclo de vida** — e merece uma máquina de estados. As regras de transição precisam ser definidas.

Não existe em nenhuma

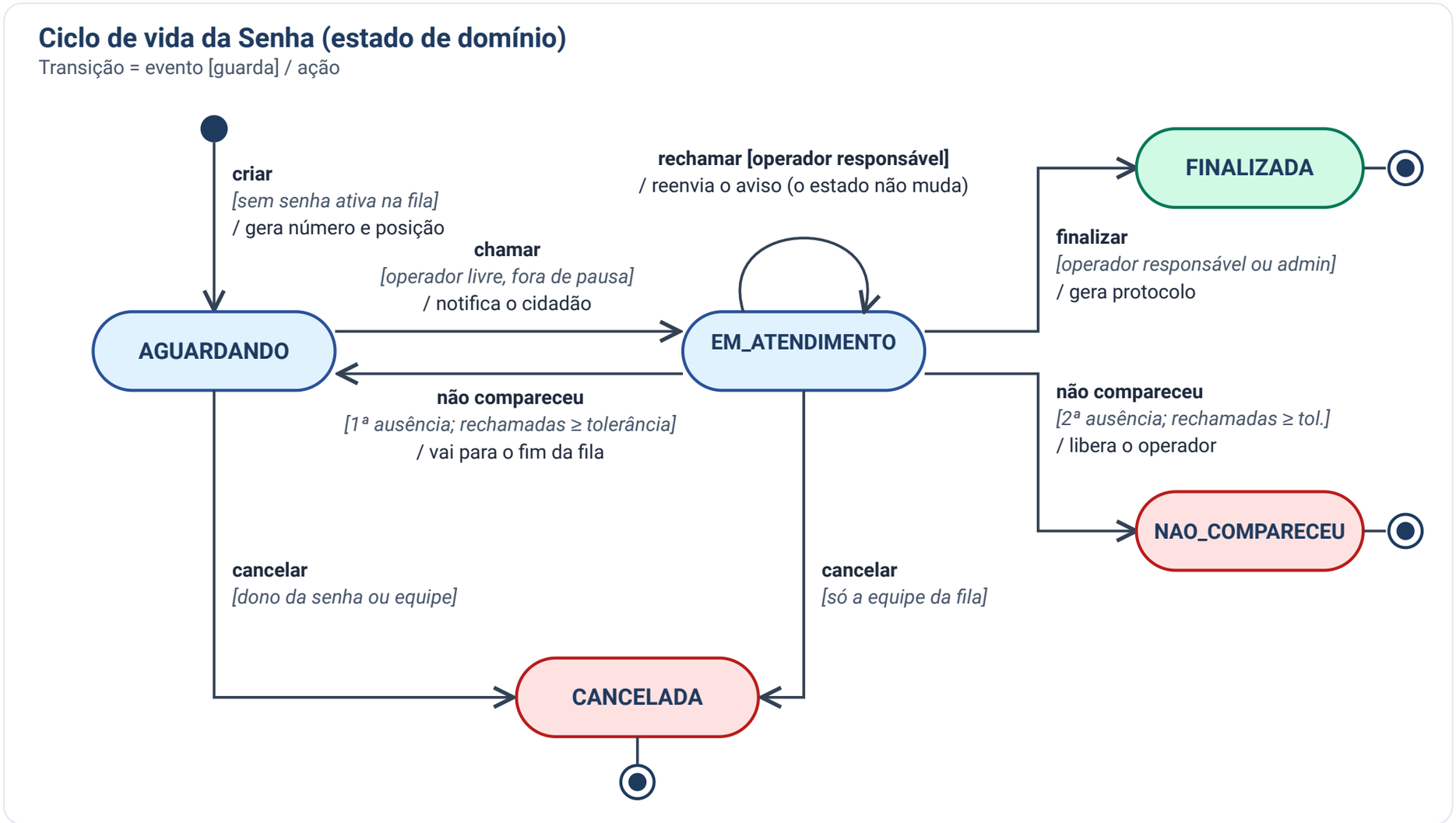
Provavelmente o sistema **não precisa** do diagrama. Não force.

Estado é **situação**, não ação: "Em atendimento", não "Atender". A seta é rotulada com o **evento** que dispara a transição.

📖 FOWLER · Cap. 10 (p. 110)

EXEMPLO: PROJETO VEZ

A Senha tem cinco estados, e só três terminam o ciclo



📖 Exemplo: projeto Vez — `docs/06-api.md` (Apêndice A) e `SenhaService.js`. FOWLER · Cap. 10.

TABELA DE TRANSIÇÕES

Cada seta vira uma linha e cada linha vira um teste

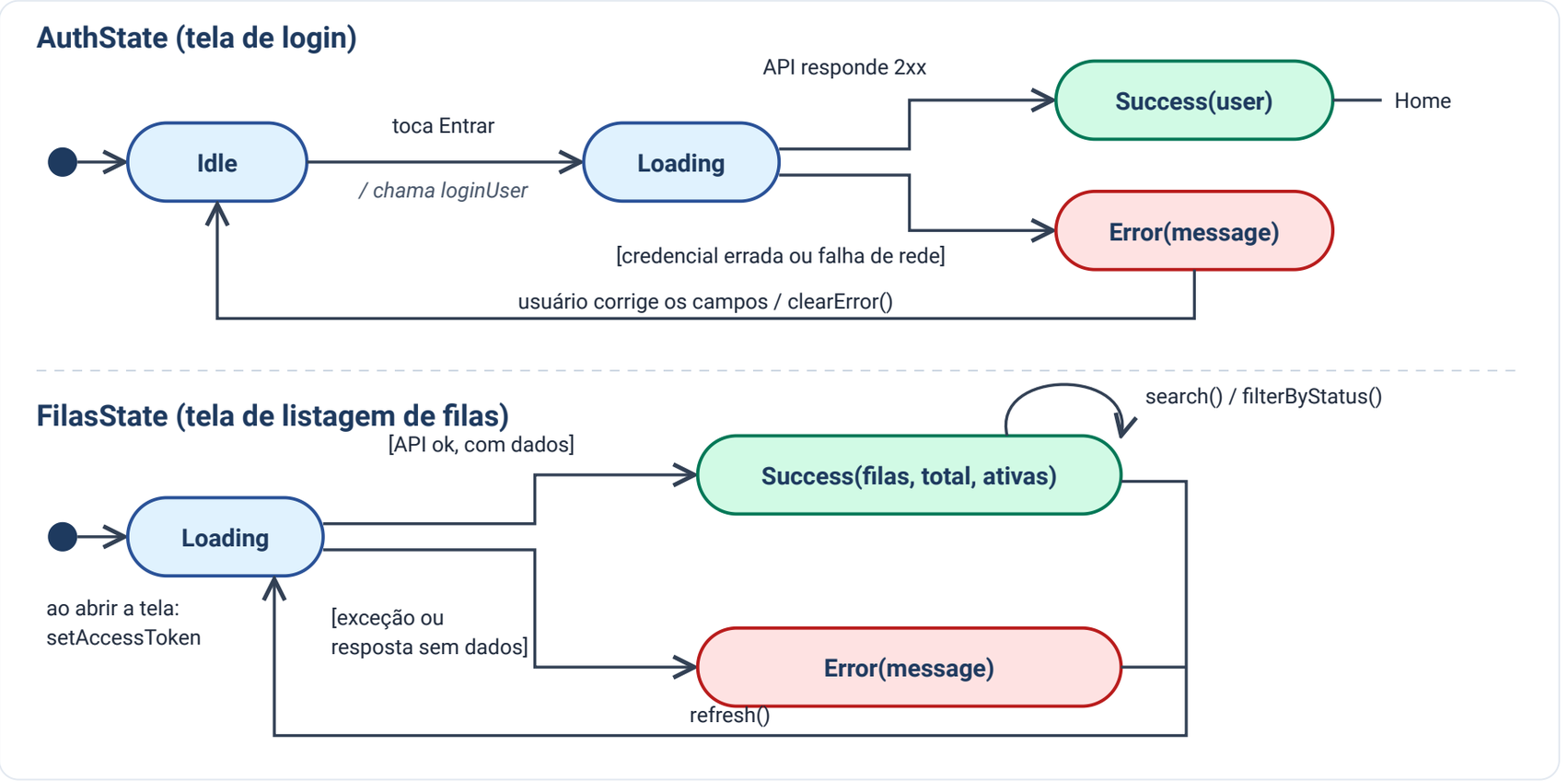
Origem	Evento [guarda]	Destino
(inicial)	criar [sem senha ativa na fila]	AGUARDANDO
AGUARDANDO	chamar [operador livre]	EM_ATENDIMENTO
EM_ATENDIMENTO	finalizar [operador responsável]	FINALIZADA
EM_ATENDIMENTO	não compareceu [2ª ausência]	NAO_COMPARECEU

> Teste também o que é **proibido**: finalizar uma senha `AGUARDANDO` deve dar 409.

📖 Exemplo: projeto Vez — `AusenciaService.js`. A tabela completa tem 8 linhas.

DOIS NÍVEIS

Estado de domínio e estado de tela usam a mesma notação

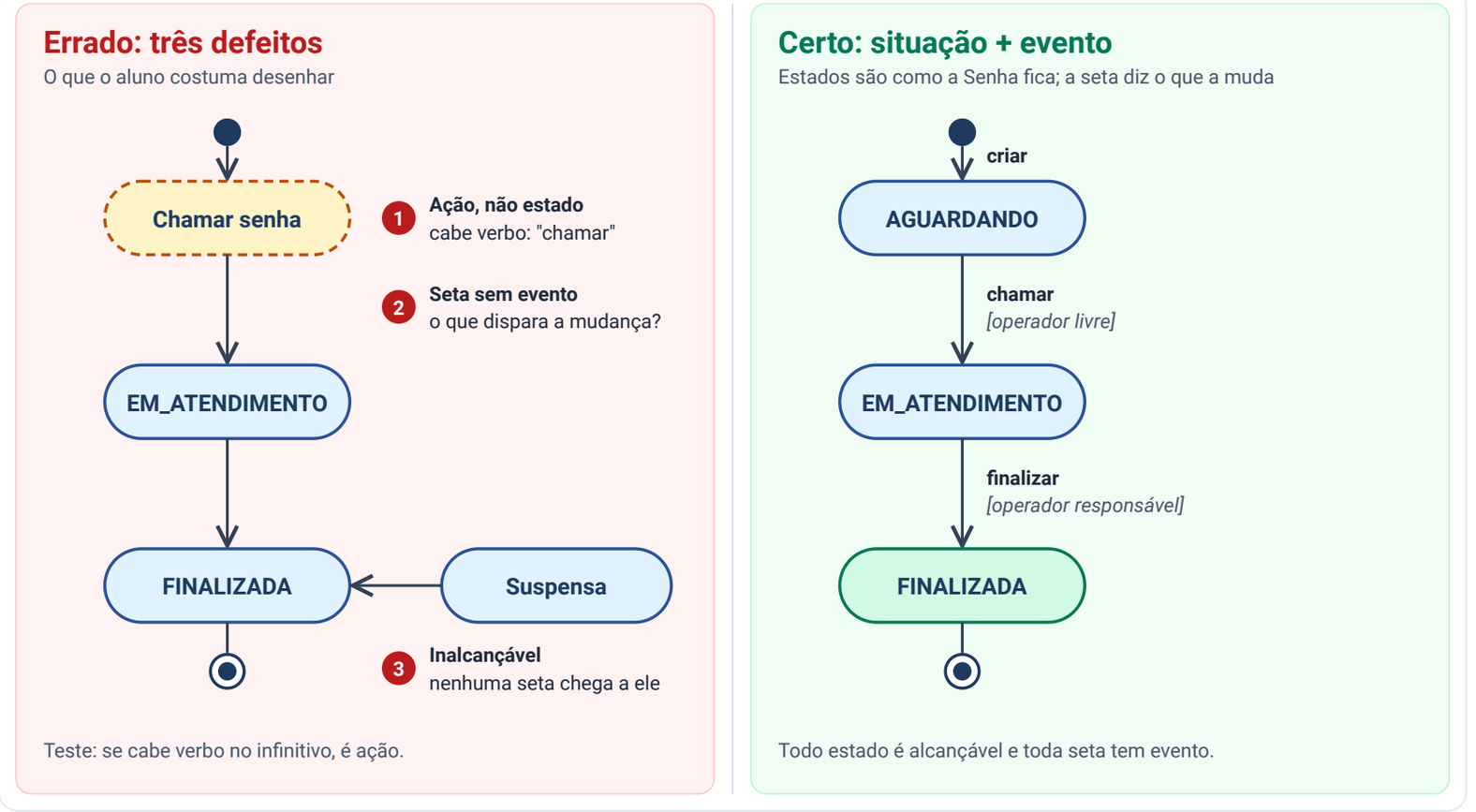


> `Senha` vive no banco por dias. `Loading` vive milissegundos. Pergunte: o estado é de **quem**?

📖 Exemplo: projeto Vez — `AuthViewModel.kt` e `FilasViewModel.kt` (app Android).

ERRO COMUM E EXERCÍCIO

Estado não é ação, e toda seta tem evento



Exercício. O evento é `finalizar`. De quais estados da Senha ele é possível?
Resposta: só de `EM_ATENDIMENTO`; dos demais, 409.

📖 Exemplo: projeto Vez. FOWLER · Cap. 10 (p. 110).

AULA 37

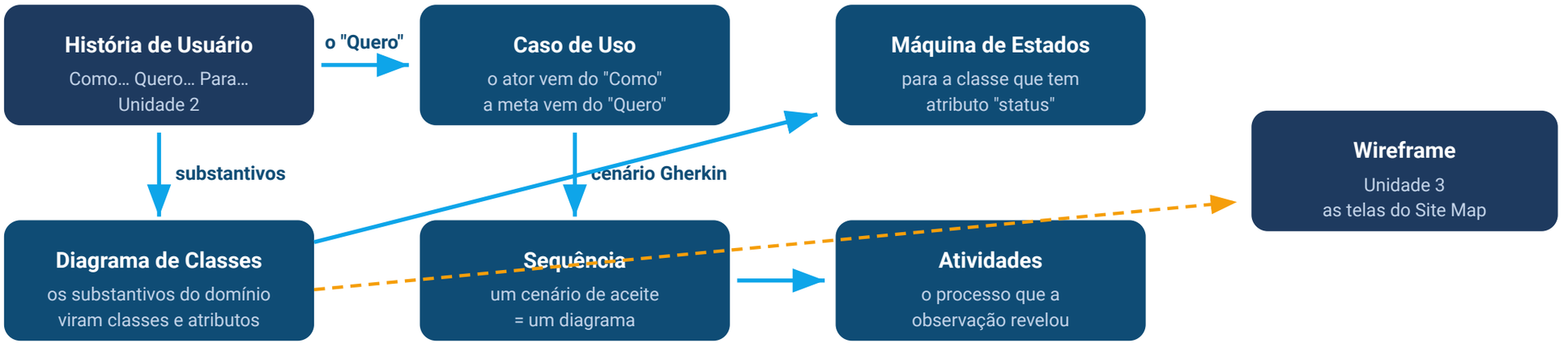
Fechando a malha

RASTREABILIDADE

De onde sai cada diagrama

De onde sai cada diagrama

Nenhum diagrama nasce do nada. Se você não sabe de qual requisito ele veio, provavelmente inventou escopo.



verificação cruzada: todo dado exibido no wireframe existe como atributo de alguma classe?
e toda classe do modelo aparece em alguma tela — ou é estrutura interna justificada?

Aula 37 — revisão cruzada UML × protótipo × requisitos

É o momento de fechar a malha. Quatro perguntas que a monitoria do Milestone 2 vai fazer:

1. Toda US tem caso de uso? 2. Todo dado da tela existe no modelo de classes?
3. Todo cenário principal tem sequência? 4. Toda classe com "status" tem máquina de estados?

SOMMERVILLE 10ed · §5.2 a §5.4 · FOWLER, UML Essencial, *cap. 2 · rastreabilidade: PRESSMAN 9ed · §7.2.6*

📖 SOMMERVILLE 10ed · §5.2 a §5.4 · rastreabilidade: PRESSMAN 9ed · §7.2.6

RASTREABILIDADE · CASO VEZ

Uma funcionalidade, seis elos: retirar senha

Malha do Vez: retirar senha, do requisito ao teste

Cada elo cita o anterior pelo ID. Se você não sabe de onde um diagrama veio, ele é órfão.

V-RF-01 Requisito	O cidadão retira uma senha em uma fila ativa da instituição. 06-api.md §8.1 · POST /filas/{filaId}/senhas
detalha	
US-V01 História de usuário	Como cidadão, quero retirar uma senha, para esperar sem ficar de pé no balcão. Unidade 2
vira a meta	
UC06 Caso de uso	Retirar senha em fila (ator: Cidadão). Diagrama de casos de uso
substantivos	
Classes Diagrama de classes	Fila (ativa) 1 — 0..* Senha (numero, posicao, status). Substantivos do requisito
realiza	
V-SQ-01 Sequência	criar(filaId, qrCode?) → Senha {AGUARDANDO}; retorno 201. Cenário principal do caso de uso
verifica	
V-CT-01 Cenário de teste	Dado fila ativa e sem senha ativa, quando retira, então AGUARDANDO + 201. Também V-CT-02: já tem senha ativa → 409

Dados do Vez (docs/06-api.md §8.1, docs/diagramas/api-uml-1, -6, -10). IDs V-… são didáticos, escolhidos para esta aula.

📖 Exemplo: projeto Vez — docs/06-api.md §8.1 e diagramas api-uml-1, -6, -10 · IDs V-… didáticos · PRESSMAN 9ed · §7.2.6

PANORAMA

Cada diagrama responde a uma pergunta

Qual diagrama responde a qual pergunta? Exemplo: Vez

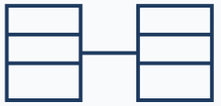
Cinco diagramas, cinco perguntas, um só sistema.



Casos de uso

Quem faz o quê?

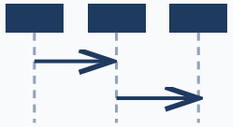
No Vez
Cidadão retira senha; Operador chama senha.



Classes

Como as peças se relacionam?

No Vez
Instituição 1—0..* Fila 1—0..* Senha.



Sequência

Em que ordem as peças conversam?

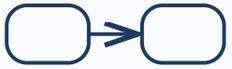
No Vez
Cidadão → API → SenhaService → repositório.



Atividades

Qual o fluxo, com decisões?

No Vez
Fila ativa? Já tem senha ativa? QR válido?



Estados

Em que estados a coisa vive?

No Vez
Senha: AGUARDANDO → EM_ATENDIMENTO → FINALIZADA.

Os outros nove diagramas da UML existem, mas não fazem parte desta disciplina.

📖 Exemplo: projeto Vez — docs/06-api.md · FOWLER · Cap. 1

ANÁLISE DE IMPACTO

Com a malha, mudar um requisito vira roteiro

O que acontece quando muda um requisito

A malha vira mapa de impacto: a mudança desce a cadeia, e cada elo diz o que rever. Mudança hipotética, para estudo.

V-RF-01 Requisito	MUDOU: máx. 1 senha ativa por instituição (hoje, por fila).	ALTERADO
US-V01 História	Critério de aceite novo: recusar se já tem senha em outra fila.	REVISAR
UC06 Caso de uso	Novo fluxo alternativo: cidadão já tem senha ativa na instituição.	REVISAR
Classes Diagrama	Fila e Senha já se ligam à instituição; confirmar se falta algo.	CONFERIR
V-SQ-01 Sequência	Novo fragmento alt [já tem senha ativa na instituição] → 409.	REVISAR
V-CT-02 Teste	V-CT-02 (409 na mesma fila) é reescrito; V-CT-03 é novo.	TESTAR

Sem IDs ligando os elos, ninguém sabe o que revisar: o erro aparece depois, como defeito.

📖 Mudança hipotética sobre o exemplo do Vez (docs/06-api.md §8.1)

MATRIZ DE RASTREABILIDADE

Célula vazia na matriz é defeito à vista

Requisito	US	UC	Classes	Seq.	Teste
V-RF-01 Retirar senha	US-V01	UC06	Fila, Senha	V-SQ-01	V-CT-01, 02
V-RF-02 Chamar/finalizar	US-V04	UC24	Senha	V-SQ-03	**vazio**
V-RF-03 Cancelar senha	US-V05	UC09	Senha	**vazio**	V-CT-04
V-RF-99 *(didático)*	US-V99	**vazio**	—	—	—

Requisito sem caso de uso: a funcionalidade sumiu na modelagem.

Diagrama órfão: nenhum requisito o pede; é escopo inventado.

📖 Exemplo didático sobre o Vez — docs/01-visao-geral.md §4 e docs/06-api.md §8.1

CHECKLIST

Antes da monitoria do Milestone 2

#	Verificação	O que o defeito indica
1	Toda US tem caso de uso?	Funcionalidade que sumiu na modelagem
2	Todo dado da tela existe como atributo?	A tela promete o que o modelo não entrega
3	Todo cenário principal tem sequência?	Ninguém pensou em como aquilo funciona por dentro
4	Toda classe com status tem máquina de estados?	As regras de transição não foram definidas
5	Toda classe aparece em alguma tela ou sequência?	Escopo inventado, ou estrutura interna não justificada

> A monitoria é o momento de descobrir o que falta — não a apresentação.

FONTES

Onde estudar

Tópico	Fonte
Os três modos de usar UML	FOWLER · Cap. 1 (p. 25) e Cap. 2 (p. 39)
Casos de uso, «include», «extend»	FOWLER · Cap. 9 (p. 104) · SOMMERVILLE 10ed · §5.2.1
Classes — básico e avançado	FOWLER · Cap. 3 (p. 52) e Cap. 5 (p. 77)
Generalização e agregação	SOMMERVILLE 10ed · §5.3.2 e §5.3.3 · VALENTE · §4.3
Sequência	FOWLER · Cap. 4 (p. 67) · SOMMERVILLE 10ed · §5.2.2
Atividades	FOWLER · Cap. 11 (p. 118) · VALENTE · §4.6
Máquina de estados	FOWLER · Cap. 10 (p. 110) · SOMMERVILLE 10ed · §5.4
Notação normativa	OMG · *UML* 2.5.1 · omg.org/spec/UML — em inglês

📖 A unidade mais bem servida do semestre: o Fowler cobre os cinco diagramas com capítulo e página conferidos.

UNIDADE 4 · ENCERRAMENTO

Atividade ES-04

Revisar um diagrama de classes defeituoso e modelar o seu projeto em UML.

Entrega individual pelo AVA · 100 pontos